The Continuous p-Dispersion Problem in Three Dimensions

Sanjay Manoj[1] and Melkior Ornik[1]
[1] *University of Illinois Urbana-Champaign*

Abstract: [!][315 words; C&OR wants about 250] The Continuous p-Dispersion Problem (CpDP) with boundary constraints asks for the placement of a fixed number of points in a compact subset of Euclidean space such that the minimum distance between any two points, as well as the points and the boundary of this compact set is maximized. This problem finds applications in facility placement, communication network design, sampling theory, and particle simulation; however, finding optimal solutions is NP-hard and existing algorithms focus on providing approximate solutions in two-dimensional space. In this paper, we introduce an almost-everywhere differentiable optimization model and global optimization algorithm for approximating solutions to the CpDP with boundary constraints in convex and non-convex polyhedra with respect to any norm-induced metric in a three-dimensional Euclidean space. Our algorithm generalizes two-dimensional dispersion techniques to three dimensions by leveraging orientation, exact metric distances from a point to the faces, edges and vertices of the boundary, and a ray-casting procedure for point-in-polyhedron testing, enabling optimization in arbitrary convex and non-convex three dimensional polyhedra. For any polyhedral norm these distances, the points realising them, and their gradients follow in closed form from the vertices of the dual unit ball, so a metric enters the algorithm as a finite set of vectors rather than as a new derivation. We validate the proposed algorithm by comparing with analytical optima where available and empirical benchmarks, observing close agreement with optimal solutions and improvements over empirical benchmarks. Given the same wall time as the general-purpose solvers IPOPT and SLSQP, it finds a larger radius in 32 and 25 of 45 test cases respectively, by up to a factor of 2.6, and a smaller one in none; where the optimum can be certified, our solutions attain it. In the eight multi-connected polyhedral domains of Stoyan and Yaskov, it packs more spheres of radius one than published in seven, by up to seven, and as many in the eighth.

Review copy: colour key.
Blue text: new or rewritten since the last commit, mostly drafted by Claude; read for voice.
Yellow: a claim or number to verify.
Orange: a framing or reporting decision that is yours to make.
Pink: to fix before the C&OR conversion; bracketed pink notes say why.

I. INTRODUCTION

In 1987, Kuby [1] proposed the p-Dispersion Problem, asking to locate p facilities on a network so that the minimum separation distance between any pair of open facilities is maximized. Later in 1995, Drezner and Erkut [2] formalized the p-Dispersion Problem, asking for the placement of p points from a subset of $\mathbb{R}^n$ such that the minimum distance between pairs of points is maximized. Subsequently, the Continuous p-Dispersion Problem (CpDP) with boundary constraints adds the requirement that a dispersion point must be sufficiently far away from the boundary of the subset of $\mathbb{R}^n$, also often referred to as a container. The p-Dispersion Problem is interesting because it formalizes the inherent trade-off between coverage and separation, to find how a limited number of points can be optimally distributed over a domain. The p-Dispersion problem is provably NP-complete for the general container [3] in the number of dispersion points p.

Existing work solving the p-Dispersion problem primarily focuses on low-dimensional containers with special geometric properties. For instance, repulsion-based heuristics have been applied to circular containers [4] and non-convex two-dimensional polygons [5]. Other approaches include a 2/3-approximation algorithm for rectilinear polygons [3] and a Tabu Search Global Optimization (TSGO) algorithm for general planar containers [6]. In higher dimensions, work is severely limited: the nonlinear model of Drezner and Erkut [2] is formulated in a way that extends to $\mathbb{R}^n$ but is demonstrated only in the unit square, while Kazakov et al. [7] developed a grid-based sphere-packing algorithm that struggles with complex, non-convex three-dimensional boundaries. Stoyan and Yaskov [8] pack congruent spheres into polyhedral domains with holes, the decision version of the Euclidean problem: how many spheres of a given radius fit.

Compared to previous work, the goal of this paper is to approximate elements of the solution set for the CpDP within general polyhedral containers with respect to any norm-induced metric in $\mathbb{R}^3$ [6]. The TSGO algorithm proposed by Lai et al. [6] is difficult to adapt to higher dimensions because it is built around two-dimensional logic. Their model relies on measuring distances to the edges of a polygon and uses repeated point-in-polygon tests to check for constraint violations. We extend this approach to solve the CpDP for general polyhedral containers in three dimensions. Our method uses a ray-casting procedure for point-in-container testing and replaces their edge-based distance checks with a formulation that calculates the distance to the faces, edges, and vertices of the three-dimensional boundary. Those distances are exact in any norm-induced metric, not only the Euclidean one: as

developed in Section III A, point-to-segment distance is a one-dimensional convex problem for every norm, and for a polyhedral norm both it and the point-to-plane distance reduce to finite evaluations over the vertices of the dual unit ball, which also supply the gradients the solver needs.

Solving the CpDP in three dimensions reflects how spatial dispersion is performed in real-world situations where objects have volume and operate in bounded physical space, such as collision avoidance for autonomous agents [9] or transmitter placement in complex indoor environments [10]. Importantly, the mathematical abstractions based on [6] established in this work are designed to be extensible to general n-dimensional containers. While the primary challenge in such a transition lies in the geometric modeling of arbitrary high-dimensional polyhedra, containers with uniform constraints, such as an n-dimensional hypercube with limits restricted between 0 and 1, remain straightforward to model within this framework. We build on the work from Lai et al. [6] by defining a feasibility-residual function to approximately solve the CpDP with boundary constraints for convex and non-convex polyhedral containers in $\mathbb{R}^3$. A feasibility-residual function $E(x) = 0$ if and only if x is feasible solution, and $E(x) > 0$ otherwise. To minimize this function, we utilize the Tabu Search heuristic [11] which improves upon standard local searches by marking previously visited configurations as "tabu" to prevent cycling and encourage the exploration of more optimal regions of the solution space [6].

We also generalize the Continuous p-Dispersion Problem by allowing the distance between points and the boundary to be measured using a metric. This allows the algorithm to handle separation constraints defined by general shapes—like cubes or octahedra—rather than being limited to standard spherical constraints. This change broadens the types of dispersion problems the algorithm can solve and provides a single framework for optimizing point layouts under different geometric requirements. We validate our algorithm's accuracy by comparing its results against optimal results in the unit cube proven by Schaer [12] and the best-known values of Packomania [13], measuring the relative deterioration to show how closely our algorithm approaches global maxima. For non-rectangular containers, we benchmark our results against the best-known packings for the unit tetrahedron provided by Kazakov et al. [7], against optima certified with Gurobi, and against the sphere counts Stoyan and Yaskov [8] report for their eight domains with holes, seven of which we exceed. We compare with the general-purpose solvers IPOPT and SLSQP at equal wall time, report results for three non-convex polyhedra of our own design, and provide visualizations for our best-achieved configurations in several containers. The centres of every configuration we report are published with a standalone script that checks them.

Basic Notation: We use the following standard notation throughout this paper. Let $d(\cdot,\cdot)$ denote a metric on $\mathbb{R}^n$ following the usual definition [14]. The distance between an element $x \in \mathbb{R}^3$ and a nonempty subset $X \subseteq \mathbb{R}^3$ is given by $d(x, X) = \inf\{d(x, y) : y \in X\}$. The *open ball* and *closed ball* of radius r centered at x are defined as $B_r^o(x) = \{y \in \mathbb{R}^3 : d(x, y) < r\}$ and $B_r^c(x) = \{y \in \mathbb{R}^3 : d(x, y) \le r\}$, respectively. For any positive integer n, let $[n] = \{1, \ldots, n\}$. Finally, for a subset $X \subseteq \mathbb{R}^3$, ∂X denotes the boundary of X, $X^c = \mathbb{R}^3 \setminus X$ is the complement of X, and $X^o = X \setminus \partial X$ is the interior of X.

II. PROBLEM FORMULATION

The Continuous p-Dispersion Problem (CpDP) asks to place p points, i.e., *sites*, within a compact subset, i.e., a *container*, in $\mathbb{R}^n$ so that the minimum pairwise distance between the sites is maximized [2]. A variation on the CpDP includes "boundary constraints" which asks that the minimum distance between any two points, as well as the points and the boundary of this compact set be maximized [3]. Most existing algorithms that solve the CpDP do so in the setting of bounded, polygonal containers in two dimensions, possibly with holes. In the three dimensions, the natural extension is to consider bounded containers which are polyhedra with polyhedral holes. A *polyhedron* is a closed and bounded solid C whose boundary ∂C can be covered by finitely many planes [15]. A *polyhedral container* is a polyhedron C with holes $U_1, \ldots, U_k \subseteq C$ which are also polyhedra.

We denote $C = (V, \mathcal{E}, \mathcal{F})$ where V is the set of vertices of C, $\mathcal{E}$ is the set of edges of C and $\mathcal{F}$ is the set of faces of C; the script letters distinguish these sets from the energy E of Equation 3.6 and from an individual face F. We require that the interior of a polyhedral container be nonempty, therefore providing the polyhedral container with a consistent orientation with inward normals [16]. We write $x \in C$ for an element $x \in \mathbb{R}^3$ inside the polyhedron but outside its holes. An element $x \notin C$ is located outside the polyhedron or inside the polyhedral container's holes.

We now state the Continuous p-Dispersion Problem (CpDP) [2] with boundary constraints for polyhedral containers with respect to a metric d on $\mathbb{R}^3$ induced by a norm (i.e., $d(x, y) = \|x - y\|$). The metrics of interest (ℓ_1, ℓ_2 and ℓ_∞) are all of this form, so this costs no generality in practice. Expressing d through a norm supplies two useful properties: *translation invariance*, $d(x + z, y + z) = d(x, y)$, and *absolute homogeneity*, $\|\lambda v\| = |\lambda| \, \|v\|$ for all $\lambda \in \mathbb{R}$. Homogeneity is what renders the norm comparable to the Euclidean metric, and allows for the almost-everywhere differentiability argument; the metric $d(x, y) = \sqrt{\|x - y\|_2}$ induces the usual topology on $\mathbb{R}^3$ but is not homogeneous.

Problem: Let $p \in \mathbb{N}$ and $C \subseteq \mathbb{R}^3$ be a polyhedral

container. We solve the nonlinear optimization problem:

$$\begin{aligned} \max_{c_1,\ldots,c_p} \quad & D \\ \text{subject to} \quad & c_1,\ldots,c_p \in C, && (i) \\ & d(c_i,c_j) \geq 2D \text{ for all distinct } i,j \in [p], && (ii) \\ & d(c_i,\partial C) \geq D \text{ for all } i \in [p]. && (iii) \end{aligned}$$

Note that we will interpret the value of D to be the radius of the ball centered at each dispersion point c_i. For a feasible solution $(c_1,\ldots,c_p,D) \in (\mathbb{R}^3)^p \times \mathbb{R}$ to the nonlinear optimization problem, we refer to the elements $c_i \in \mathbb{R}^3$ as *dispersion points* and $D \in \mathbb{R}_{>0}$ as the *radius* of the solution. A *configuration* (of dispersion points) X refers to the tuple $X = (c_1,\ldots,c_p) \in (\mathbb{R}^3)^p$ for a given feasible solution; the ordering is immaterial to the problem but fixes the indexing used by the sums below.

These constraints enforce three geometric requirements: (i) the *containment constraint* ensures each point c_i lies within the container; (ii) the *dispersion constraint* requires mutual separation, meaning $B_D^o(c_i) \cap B_D^o(c_j) = \emptyset$ for all distinct $i,j \in [p]$; and (iii) the *boundary constraint* keeps points clear of the boundary, ensuring $B_D^o(c_i) \cap \partial C = \emptyset$ for all $i \in [p]$.

From the assumption that the interior of the container C is nontrivial, a feasible solution $(c_1,\ldots,c_p,D)$ always exists [5].

The CpDP is hard to attack naively because the feasible set of the nonlinear program is non-convex and admits many local optima. As a result, standard nonlinear optimizers often get caught in local minima. Additionally, the number of distance constraints to compute is $\binom{p}{2} = \mathcal{O}(p^2)$, and with minimum distance bounds, it is highly likely that a random initial configuration will violate the dispersion constraint (ii).

III. OPTIMIZATION MODEL

In this section we introduce a unified feasibility-residual function used to solve the CpDP with boundary constraints by modeling constraint violation in two parts: *dispersion violation* and *container violation*. Dispersion violation stems from dispersion points being located too close together; namely, when $d(c_i,c_j) < 2D$ for some fixed $D > 0$, constraint (ii) is violated. Container violation is a result of dispersion points being located too close to the boundary of the container; namely, when $d(c_i,\partial C) < D$ for some fixed $D > 0$, constraint (iii) is violated.

By combining these two components into a single feasibility-residual function, we can address inherent geometric complexities associated with any container. In general, the feasible region for the solution set to the CpDP is non-convex. Moreover, as the number of dispersion points p increases, the feasible region will contain many local optima [6].

Lai et al. [6] proposed a novel framework for the two-dimensional CpDP by unifying dispersion and boundary constraints into a single, non-negative feasibility-residual function, E_D. They conceptualized constraint violations as *energy*, where the total energy of a configuration corresponds to its distance from feasibility. For a configuration of points $X \in (\mathbb{R}^2)^p$ and fixed $D \geq 0$, this function is defined as $E_D(X) = \sum_{1\leq i<j\leq p} \ell_{ij}^2 + \sum_{i=1}^{p} O_i$ where ℓ_{ij} is the dispersion penalty between points c_i and c_j, and O_i is the boundary penalty for point c_i. A crucial property of this formulation is that $E_D(X) = 0$ if and only if (X,D) is a feasible solution. Because E_D is almost-everywhere differentiable, it allows for the application of efficient gradient-based optimization methods, such as the Limited-memory Broyden-Fletcher-Goldfarb-Shanno (L-BFGS) algorithm [17]. This approach proved exceptionally powerful, enabling the rapid convergence to local optima from random initial configurations.

Extending the optimization model from Lai et al. [6] to three dimensions introduces several significant challenges. While the dispersion penalty term is easy enough to compute in any finite number of dimensions, computing the boundary penalty term in three dimensions becomes computationally complex. First, Lai et al. [6] defines their boundary constraint term in a manner which heavily relies on planar geometry where counter-clockwise traversal of closed polygonal loops provides a canonical way to distinguish the interior from the exterior. We generalize boundary orientation of polygonal containers from two-dimensional vertex traversal to three dimensions by orienting the faces of the polyhedron using inward normals. By assigning an inward-pointing normal to each face, we explicitly define the interior half-space of the polyhedron. In our algorithm, these normals serve as the directional basis for calculating distance-to-boundary terms for dispersion points and performing feasibility checks.

Additionally, point-in-container testing is fundamental to the boundary constraint term, as it dictates whether to apply a repulsive force to maintain interiority or an attractive force to recover feasibility from the exterior. While point-in-polygon testing admits efficient deterministic routines [18], no such efficient subroutines are devised for three-dimensional containers that scale well with the number of vertices. Consequently, we utilize a ray-casting-based point-in-polyhedron procedure [19] together with the polyhedron's orientation to obtain distances to the boundary of complex or non-convex polyhedral containers. The ray-casting procedure draws several rays in random directions from a given dispersion point and counts the number of faces each intersects. The directions are drawn once per run and held fixed for its duration, which is what keeps the model well posed: re-drawing them at every evaluation would make E a random function of X, so the solver would be minimizing a stochastic objective and gradient-based descent would be unjustified. Fixing them per run keeps E deterministic in X while preventing any single alignment from biasing every run. If a given ray

intersects an even number of faces, the dispersion point is categorized as exterior; an odd number of intersections identifies the point as interior. Because rays may occasionally strike a vertex or edge leading to numerical ambiguity, we cast k rays and take a majority vote. A small k suffices, and does not need to grow with the container. For a query point at distance r from the nearest edge, the directions that resolve a crossing ambiguously occupy an angular band of width $O(\epsilon/r)$ about a measure-zero arc, so a single ray misclassifies with probability $\pi \sim c\,|\mathcal{E}|\,\epsilon/r$ and a k-ray majority fails with probability $O((4\pi)^{k/2})$. The edge count enters only inside a logarithm, giving $k = O(\log|\mathcal{E}|)$; at $\epsilon \sim 10^{-10}$ and $|\mathcal{E}| \sim 200$ we use $k = 8$.

That bound degrades as $r \to 0$, which is precisely the regime the dispersion points occupy at convergence, when they are pressed against the walls. Rays are the wrong instrument there, so we do not use them: a point within a tolerance ϵ_{plane} of some face is classified as interior before any ray is cast. The classification is then exact rather than probabilistic in the only regime where it is delicate. The Active Face rule of the next section is made inclusive to the same ϵ_{plane}, so that a point admitted as interior by this test cannot simultaneously fail to see the very wall that admitted it — were the two tolerances to disagree, a point resting on a wall would lose that wall from its active set and report a clearance measured to some distant feature. These geometric tools provide the foundation needed to extend the underlying optimization model to three dimensions.

With this geometric framework in place, we now adapt the optimization model presented in Lai et al. [6] to three dimensions. The *energy of a solution* $(c_1, \ldots, c_p, D)$ will be measured by its quantification of constraint violation. Recall that high energy solutions have a high degree of constraint violation and low energy solutions have a low degree of constraint violation.

We first record the regularity facts used to establish that the feasibility-residual function is almost-everywhere differentiable. Both penalty terms below are built from metric distances composed with the same one-dimensional map, so it is convenient to isolate their common structure.

Lemma 1. *Let d be induced by a norm on $\mathbb{R}^3$. Then*

(a) $(x, y) \mapsto d(x, y)$ is Lipschitz on $\mathbb{R}^3 \times \mathbb{R}^3$ with respect to the Euclidean structure;

(b) for any nonempty $S \subseteq \mathbb{R}^3$, $x \mapsto d(x, S)$ is 1-Lipschitz;

(c) for nonempty $S_1, \ldots, S_m \subseteq \mathbb{R}^3$, the pointwise minimum $x \mapsto \min_{k\in[m]} d(x, S_k)$ is 1-Lipschitz;

(d) $g(t) = (\max\{0, t\})^2$ belongs to $C^1(\mathbb{R})$.

Consequently, if f is locally Lipschitz then $g \circ f$ is differentiable almost everywhere.

Proof. (a) The triangle inequality gives $|d(x, y) - d(x', y')| \le d(x, x') + d(y, y')$. Since all norms on $\mathbb{R}^3$ are equivalent, there is $C > 0$ with $d(u, v) \le C\,\|u - v\|_2$, so d is Lipschitz with respect to the Euclidean structure. (b) For every $s \in S$, $d(x, S) \le d(x, y) + d(y, s)$; taking the infimum over s and exchanging x and y yields $|d(x, S) - d(y, S)| \le d(x, y)$. (c) A minimum of finitely many 1-Lipschitz functions is 1-Lipschitz. (d) $g'(t) = 2\max\{0, t\}$ is continuous, so squaring removes the kink that $\max\{0, t\}$ has at the origin. For the final claim, f is differentiable almost everywhere by Rademacher's Theorem [20], and the chain rule applies wherever it is, since g is continuously differentiable. □

First, we define the penalty term for dispersion violation. Let

$$\ell_{i,j} = \max\{0, 2D - d(c_i, c_j)\}, \tag{3.1}$$

hence we get $\ell_{i,j} = 0$ if and only if $d(c_i, c_j) \ge 2D$. Therefore the total pairwise dispersion penalty $\sum_{1\le i<j\le p} \ell_{i,j}^2$ equals 0 if and only if all the dispersion points are sufficiently far from each other. Moreover $\ell_{i,j}^2 = g(2D - d(c_i, c_j))$ with g as in Lemma 1(d), and $(c_i, c_j) \mapsto 2D - d(c_i, c_j)$ is Lipschitz by Lemma 1(a). Hence $\ell_{i,j}^2$ is differentiable almost everywhere for all i, j.

A. Distances under a polyhedral norm

The boundary penalty measures the distance from a dispersion point to three kinds of feature: the container's vertices, its edges, and its faces. Only the first is immediate. We record here the construction used for the other two, because it is what makes the model metric-agnostic: a norm enters the algorithm only through a finite set of vectors, and one argument then settles the distance, the point realising it, and the gradient, for segments and planes alike.

Throughout, $\|\cdot\|$ is a norm on $\mathbb{R}^3$, $d(x, y) = \|x - y\|$, and $\|z\|_* = \max\{\langle z, u\rangle : \|u\| \le 1\}$ is the dual norm. Call $\|\cdot\|$ *polyhedral* when its unit ball is a polytope; equivalently, when the dual unit ball is a polytope with vertices $g_1, \ldots, g_K$, so that

$$\|z\| = \max_{k\in[K]} \langle g_k, z\rangle. \tag{3.2}$$

Both non-Euclidean metrics used in this paper are of this kind. For ℓ_∞ the dual ball is the ℓ_1 ball, whose vertices are the six vectors $\pm e_i$; for ℓ_1 the dual ball is the ℓ_∞ ball, whose vertices are the eight sign vectors $(\pm 1, \pm 1, \pm 1)$. A further polyhedral norm — weighted, anisotropic, hexagonal — is supplied to the algorithm by giving its generators, not by deriving new geometry.

Segments. For a segment $[s, e]$ and a point x,

$$d(x, [s, e]) = \min_{t\in[0,1]} \|(x - s) - t(e - s)\|. \tag{3.3}$$

The function being minimised is a norm composed with an affine map of t, hence convex on $[0,1]$ *for every norm*. Point-to-segment distance is therefore always a one-dimensional convex problem, never a per-metric derivation; what a metric changes is only how the minimum is located. Substituting Equation 3.2 into Equation 3.3 gives

$$d(t) = \max_{k\in[K]} (\alpha_k + t\beta_k), \quad \alpha_k = \langle g_k, x-s\rangle, \ \beta_k = -\langle g_k, e-s\rangle,$$

the upper envelope of K affine functions: convex and piecewise linear. The minimum of such an envelope over $[0,1]$ is attained at an endpoint or at a kink, and every kink is a crossing of two of the lines, so the exact minimiser lies in the finite set

$$\{0,1\} \cup \left\{ \tfrac{\alpha_j-\alpha_i}{\beta_i-\beta_j} : i<j \right\} \cap [0,1],$$

and is obtained by evaluation rather than by search.

Gradients. Only α_k depends on x, with $\partial\alpha_k/\partial x = g_k$. If the minimum is attained at an endpoint, t^* is pinned there under small perturbations of x, and Danskin's Theorem gives $\partial d/\partial x = g_{k^*}$ for any k^* attaining the maximum. If instead it is attained at a crossing of lines i and j, then $t^* = (\alpha_j-\alpha_i)/(\beta_i-\beta_j)$ moves with x, and differentiating $d = \alpha_i + t^*\beta_i$ through that dependence yields

$$\frac{\partial d}{\partial x} = g_i + \beta_i \frac{g_j - g_i}{\beta_i - \beta_j}.$$

The two cases are the endpoint and interior branches of one expression; neither is specific to ℓ_1 or ℓ_∞.

Planes. Let $P = \{y : \langle N, y-q\rangle = 0\}$ and write $h = \langle N, x-q\rangle$. Then

$$d(x,P) = \frac{|h|}{\|N\|_*}, \tag{3.4}$$

and the minimum is attained at $x - su$, where $s = h/\|N\|_*$ and u is any vector with $\|u\| \le 1$ and $\langle N, u\rangle = \|N\|_*$. Indeed, for every $y \in P$ we have $\langle N, x-y\rangle = h$, so $|h| \le \|N\|_* \|x-y\|$ by definition of the dual norm, giving $d(x,P) \ge |h|/\|N\|_*$; and the stated point lies in P and realises the bound. For a polyhedral norm the maximum defining $\|N\|_*$ is attained at a vertex of the unit ball, so u is found among finitely many candidates. For ℓ_2, $\|N\|_* = \|N\|_2$ and $u = N/\|N\|_2$ is unique, and Equation 3.4 reduces to orthogonal projection.

Two features of Equation 3.4 deserve emphasis, because the Euclidean intuition fails on both. First, the orthogonal projection of x onto P is *not* a nearest point of P unless N happens to be parallel to a vertex of the unit ball, so using it in place of Equation 3.4 over-estimates the distance from a dispersion point to a slanted wall. Second, the nearest point need not be unique: the set of optimal directions

$$U^* = \{u : \|u\| \le 1, \ \langle N, u\rangle = \|N\|_*\}$$

is a face of the unit ball, and reduces to a single direction only when that face is a vertex. For ℓ_∞ with $N = e_1$, every u with $u_1 = 1$ and $\|u\|_\infty \le 1$ is optimal, so the nearest points of P fill a square rather than forming a single foot. All of them lie at the same distance from x, so Equation 3.4 is unambiguous; but they lie at different places on P, and the next definition turns on *where* they lie.

Second, before we define the boundary constraint term, we need two auxiliary definitions which are three-dimensional analogues of the definitions presented by Lai et al. [6]:

Active Face: Consider a polyhedral container C with a face F and a point c. Orient the normal vector N_F of face F inwards and denote $q_F \in F$ as the point in which N_F originates from. *A face F is active with respect to c* if either $c \in C$ and $N_F \cdot (c - q_F) \ge 0$ or if $c \notin C$ and $N_F \cdot (c - q_F) < 0$.

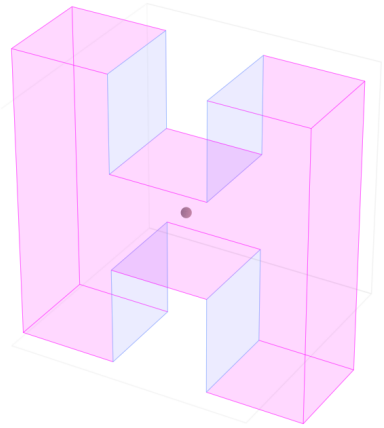


FIG. 1: All active faces (magenta) of the H-box container for point $c = (1.5, 0.5, 1.5)$ inside the container.

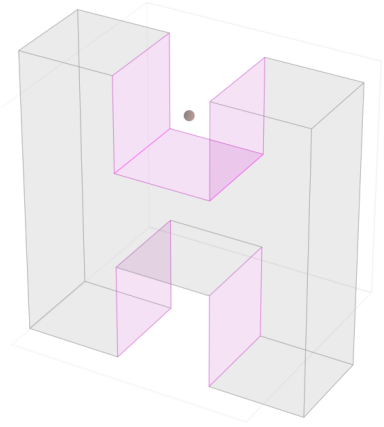


FIG. 2: All active faces (magenta) of the H-box container for point $c = (1.5, 0.5, 2.5)$ outside the container.

Fig. 1 depicts the active faces for a point inside the H-shaped container, and Fig. 2 depicts the active faces for a point outside the H-shaped container. An active face can be thought of as an oriented plane that is geometrically relevant to the point c. If c lies inside the polyhedron, then only the faces whose inward-normals minimally constrain how close c is to the boundary of the container constitute the set of active faces. Alternatively, if c lies outside the boundary of the polyhedron, an active face is blocking c from entering the polyhedron. Active faces will contribute to the boundary constraint in the feasibility-residual function.

Active Footpoint: Consider a dispersion point c and an active face F of the container C, and let P be the plane through q_F with normal N_F. A *foot* of c on P is a nearest point of P to c in the metric d; by Equation 3.4 every foot lies at distance $|\langle N_F, c - q_F\rangle| \,/\, \|N_F\|_*$ from c. We say c has an *active footpoint* h with respect to F if some foot of c on P lies on F, and we take h to be such a foot. We denote H^i to be the set of all active footpoints with respect to the dispersion point c_i.

Under the Euclidean metric the foot is unique and equals the orthogonal projection of c onto P, which recovers the construction of Lai et al. [6]. Under a general norm it is neither unique nor orthogonal, and the existential quantifier in the definition is load-bearing. Since all feet of c on P are equidistant from c, the *distance* an active footpoint contributes does not depend on which foot is named; what does depend on it is whether the named point lies on F, and hence whether F contributes to the penalty at all. Requiring only that *some* foot lie on F makes this a question about the geometry of $P \cap F$ rather than about an arbitrary selection, and it is exactly the property Lemma 2 needs.

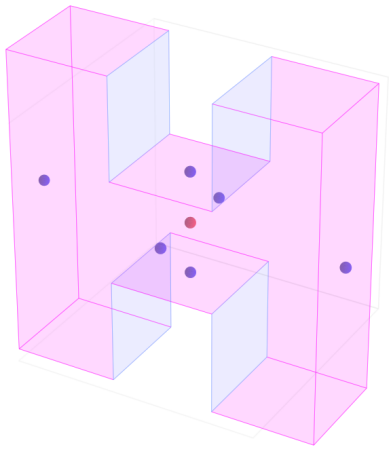


FIG. 3: All active footpoints (blue) inside the H-box container with respect to (red) point $(1.5, 0.5, 1.5)$ inside container with active faces highlighted in pink.

Fig. 3 depicts a case where the footpoints of a dispersion point are all active. Fig. 4 depicts a case where not all footpoints of a dispersion point are active. The active

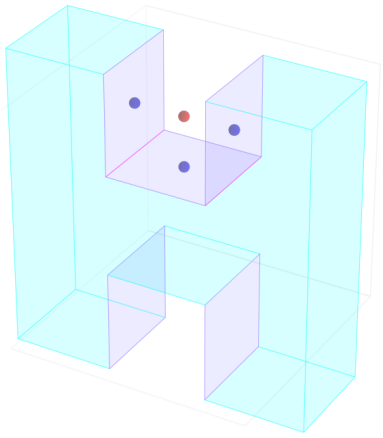


FIG. 4: All active footpoints (blue) inside the H-box container with respect to (red) point $(1.5, 0.5, 2.5)$ outside container with active faces highlighted in purple.

footpoint h of a dispersion point c allows us to measure distances between dispersion points and the minimally relevant planes of the container, thereby providing a reference for the distance between a dispersion point and the container's boundary. When a nearest point of P falls directly on the face F, F is the boundary face that governs how close c may move toward the container's boundary, and the point-to-face distance is determined by the separation between c and this footpoint. When no nearest point of P lies on F, the nearest point of F itself lies on the relative boundary of F, which is a union of edges; the penalty then reaches that part of the boundary through $\mathcal{E}$ rather than through H^i, and nothing is lost.

Orientation equips the polyhedron with coherent inward normals, making the active face selection rule an orientation-consistent generalization of the counterclockwise active-edge mechanism used in the planar formulation [6]. Unlike the two-dimensional case, where the boundary is a closed, non-self-intersecting loop of segments with an inherent cyclic ordering, the boundary of a three-dimensional container forms a polyhedral complex with no analogous ordering. The extension from active edges [6] to active faces and their associated footpoints provides the geometric machinery needed to generalize boundary interactions from vertices and edges in two dimensions to vertices, edges, and faces in three dimensions. Thus, active faces constitute the minimal and geometrically natural extension of the active-edge concept from two to three dimensions, enabling a unified treatment of boundary constraints in arbitrary polyhedral domains.

Now, we define the boundary penalty for a dispersion

point c_i as

$$O_i = \begin{cases} \left(D + 2\min\left\{d(c_i, v) : v \in V \cup H^i\right\}\right)^2 & c_i \notin C, \\ \sum_{v \in V \cup \mathcal{E} \cup H^i} \left(\max\left\{0,\ D - d(c_i, v)\right\}\right)^2 & c_i \in C. \end{cases} \tag{3.5}$$

When $c_i \in C$, the penalty accumulates the squared *shortfall* of each boundary feature's distance below D: it vanishes exactly when the open ball $B_D^o(c_i)$ meets no vertex, edge or active footpoint of the container, and otherwise grows with the depth of the violation. If the distance from c_i to a vertex, edge or footpoint exceeds D, that feature contributes nothing.

The exterior branch minimises over $V \cup H^i$ only. Omitting $\mathcal{E}$ costs nothing there: outside the container the penalty has no zero to reach, so its role is not to measure the distance to the boundary exactly but to be positive and to point back inside, and $V \cup H^i$ already determines a direction of return. Inside, by contrast, the edges are indispensable — they are what covers the case in which no foot of c_i lands on its face, as the proof of Lemma 2 shows.

When $c_i \notin C$, a penalty is imposed regardless of D, since the point violates the containment constraint (i). The squared form is bounded below by $D^2 > 0$ and has non-vanishing derivative in the distance from c_i to the container, so an exterior point is not merely penalised but actively driven back toward it; a penalty differing from the interior branch by an additive constant would leave the gradient unchanged and exert no such force.

We now define the energy function $E : (\mathbb{R}^3)^p \times \mathbb{R} \to \mathbb{R}$ where $D > 0$ is fixed and $X = (c_1, \ldots, c_p) \in (\mathbb{R}^3)^p$ by

$$E(X, D) = \sum_{1 \le i < j \le p} \ell_{i,j}^2 + \sum_{i=1}^{p} O_i. \tag{3.6}$$

Lemma 2. *For fixed $D > 0$, $O_i = 0$ if and only if $c_i \in C$ and $d(c_i, \partial C) \ge D$. Consequently $E(X, D) = 0$ if and only if (X, D) is feasible.*

Proof. *If $c_i \notin C$ then $O_i \ge D^2 > 0$. Suppose $c_i \in C$. Every term of the interior branch vanishes exactly when $d(c_i, v) \ge D$ for all $v \in V \cup \mathcal{E} \cup H^i$, so it suffices to prove the covering identity*

$$\min_{v \in V \cup \mathcal{E} \cup H^i} d(c_i, v) = d(c_i, \partial C). \tag{3.7}$$

Each feature on the left is a subset of ∂C, so the left side is at least the right. For the reverse inequality, ∂C is compact, so the infimum defining $d(c_i, \partial C)$ is attained at some y^*, lying on some face F. We claim F is active. If not, then $\langle N_F, c_i - q_F \rangle < 0$, so c_i lies strictly on the outward side of the plane of F. Points of the open segment (y^*, c_i) near y^* then lie outside C, while $c_i \in C$, so that segment meets ∂C at some z strictly between y^* and c_i. As z lies on the segment, $d(c_i, z) < d(c_i, y^*)$, contradicting the minimality of y^*.

Now either some foot of c_i on the plane of F lies on F, in which case that foot belongs to H^i and, by Equation 3.4, realises $d(c_i, F) = d(c_i, \partial C)$; or no foot does, in which case the minimiser y^* of $d(c_i, \cdot)$ over F lies on the relative boundary of F, a union of edges of F, all of which belong to $\mathcal{E}$. Either way the left side of Equation 3.7 attains $d(c_i, \partial C)$, proving the identity. Hence $O_i = 0$ exactly when $d(c_i, \partial C) \ge D$, which is constraint (iii).

The first term of Equation 3.6 vanishes precisely when $d(c_i, c_j) \ge 2D$ for all $i \ne j$, which is constraint (ii). Both terms are non-negative, so their sum vanishes only when each does. □

The first term of Equation 3.6 deals with the dispersion constraint (ii) of the CpDP, and the second term with the containment and boundary constraints (i) and (iii). When D is fixed, we write $E_D(X) := E(X, D)$.

Throughout, the quantity we report is the *radius* D, not the minimum separation. Constraints (ii) and (iii) are deliberately asymmetric in it — two points must be $2D$ apart, while a point must clear the boundary by D — because D is the radius of the largest ball that can be centred at every dispersion point without overlapping another or leaving the container. A configuration reported at D therefore has minimum pairwise distance $2D$. Comparisons against sphere-packing tables [12, 13] are radius to radius.

Equation 3.6 gives a feasibility-residual function which lifts the optimization model proposed by Lai et al. [6] from two dimensions into three dimensions. It is also differentiable almost everywhere. Fix a configuration at which no active footpoint lies on the boundary of its face. On a neighbourhood of such a configuration the active sets $H^1, \ldots, H^p$ are constant, so E is a finite sum of terms $g\left(D - d(\cdot, v)\right)$ with v ranging over a fixed set; each is locally Lipschitz by Lemma 1, hence so is E, and Rademacher's Theorem applies. Two families of configurations are excluded: those at which some footpoint meets the boundary of its face, and those at which some c_i lies on ∂C, where O_i switches branches. Each is cut out by finitely many equations, so both are finite unions of sets of codimension one and therefore have Lebesgue measure zero. Hence E is differentiable almost everywhere on $(\mathbb{R}^3)^p \times \mathbb{R}_{>0}$, which is what permits the use of gradient-based solvers such as L-BFGS [17].

We note that E is not continuous everywhere, and that both of its discontinuities come from the same source: under a sum, a single geometric constraint is witnessed by a face, its edges and its vertices at once, and the sum weights it by a multiplicity that changes as c_i moves. In the interior, a footpoint leaving its face has its contribution dropped while the corresponding edge term remains; on the unit cube at $D = 0.15$ we measure a step of 2.1×10^{-2} across such an event along a path that stays strictly inside the H-box. At the wall, the interior branch tends to the sum of $(D - d_f)^2$ over every feature within D, which exceeds the D^2 the exterior branch takes there whenever more than one feature is in range — near an edge or a corner, always. Crossing the cube's floor 0.04 from an edge, E steps from 0.0467 to 0.0225. This does

not affect almost-everywhere differentiability — the argument above is local, and applies on the open set where the active sets are constant — but it does mean a line search may encounter a discontinuity.

Aggregating the interior branch as $(\max\{0, D - \min_v d(c_i, v)\})^2$ removes both jumps at once — one constraint is then counted once, and the interior branch tends to D^2 at the wall, matching the exterior one — and makes E globally Lipschitz, at which point Lemma 1(c) gives almost-everywhere differentiability directly, with no excluded set. On the two measurements above the minimum reads 0.0225 on both sides of the wall. We retain the sum because it performed marginally better at the top of the distribution: across four container–p cells the best radius found under the minimum was between 0.975 and 1.000 times that found under the sum. The minimum was, however, the more reliable of the two per seed, raising the mean radius from 0.359 to 0.388 on the H-box at $p = 10$ and from 0.106 to 0.112 on the tetrahedron at $p = 10$, so the two aggregations trade reliability against best-case quality rather than dominating one another.

Both E and Φ_μ are differentiated in closed form rather than by finite differences. The dispersion term contributes $-4\max\{0, 2D - d(c_i, c_j)\}\,\partial d/\partial c_i$ to each point; the vertex and footpoint terms differentiate through the metric directly, using the subgradient of the norm at the displacement; the edge terms use the segment gradient of Section III A; and the face terms use $\partial\left(|h| / \|N\|_*\right)/\partial c_i = \operatorname{sgn}(h) N / \|N\|_*$, which is independent of which foot the face contributes. The derivative with respect to D accumulates alongside, so one pass returns Φ_μ, $\nabla_X \Phi_\mu$ and $\partial\Phi_\mu/\partial D$ together.

This is not merely an optimization. A finite-difference gradient over the $3p + 1$ variables costs that many extra evaluations of E, or twice that for central differences, against the single pass the closed form needs. On the unit cube we measure the analytic gradient at 20× the speed of central differences at $p = 10$ and 185× at $p = 60$, the gap widening with p exactly as the count of perturbations predicts. Just as importantly, a differencing step interacts badly with a function that is only almost-everywhere differentiable: a step that straddles a kink returns a slope belonging to neither side, injecting noise that the closed form does not have.

IV. GLOBAL OPTIMIZATION ALGORITHM

In this section we propose our global optimization algorithm which builds upon the framework introduced by Lai et al. [6]. The global optimization process is structured as a multi-start pipeline to navigate the complex, non-convex solution space. In each global iteration, the algorithm restarts by generating an initial random configuration of p points uniformly distributed within the polyhedral container. This multi-start approach is necessary because the problem's non-smooth objective function and numerous distance constraints frequently trap standard optimization tools in poor local optima [5]. As shown in Fig. 5, within each iteration, the Tabu Search procedure acts as a feasibility solver by minimizing the feasibility-residual function 3.6 to find a valid configuration given a fixed minimum distance D. Once a feasible configuration is found, a distance adjustment procedure applies the Sequential Unconstrained Minimization Technique (SUMT) [21] to iteratively maximize D while preserving the feasibility of the configuration.

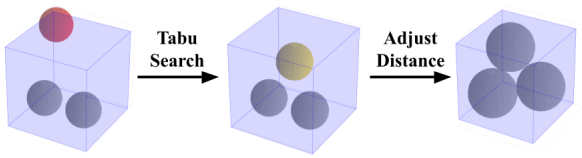


FIG. 5: Overview of the global optimization algorithm. The process demonstrates a two-stage approach: first, Tabu Search is employed to relocate a candidate sphere (red) from an initial, infeasible position to a feasible position (yellow). Second, an Adjust Distance phase maximizes the uniform radius of all spheres to achieve an optimal packing configuration.

A. Tabu Search

We employ a Tabu Search procedure [11], following the framework used by Lai et al. [6]. Tabu Search is a combinatorial heuristic designed to escape poor local configurations (high-energy) and move the system toward a better (lower-energy) arrangement of dispersion points inside the container. The core principle behind the Tabu Search is the *tabu list* which maintains a short term list of recent moves that are temporarily forbidden in subsequent iterations [11]. The tabu list ensures the algorithm does not fall into cycles of re-evaluating configurations.

A move in our setting relocates a point to a position in $\mathbb{R}^3$, and a continuous position never recurs exactly, so a list keyed on positions would forbid nothing. We therefore key an entry on the pair (index of the moved point, cell of a grid of side $\delta = D/20$ containing its destination), which makes "the same move" a statement about arriving in the same small region rather than at the same coordinates. Tying δ to D rather than to the container keeps the granularity of the memory proportional to the scale of the configuration being built.

Entries expire two ways. Each is forbidden for a tenure of 7 iterations, after which it lapses whether or not the list is full; and the list holds at most 20 entries, the least recent being evicted when it overflows. We also add the *reverse* of each accepted move, which is what prevents the search from immediately returning a point to the cell it just left. Finally, the prohibition is subject to an *aspiration criterion*: a forbidden move is taken anyway if

it yields a configuration strictly better than the best seen so far. Tabu status is a heuristic about where the search has already been, and it should never be allowed to veto an outright improvement.

The Tabu Search heuristic incentivizes long-term exploration of the solution space. The maximum number of successive moves the algorithm is allowed to explore without improving from the initial configuration is specified by a parameter β. If Tabu Search makes β iterations without improving on the last-improved configuration, then the procedure terminates. A small β enables a quicker Tabu Search, but only scans for solutions which are a few moves different from the initial configurations. A large β enables a deeper scan of neighboring solutions, but forces a longer runtime for Tabu Search. Tabu Search proves to be a powerful tool for efficiently scanning families of solutions using combinatorial optimization methods on a continuous region and transforming high-energy configurations to low-energy configurations.

For a given configuration $X = \{c_1, \ldots, c_p\}$, we quantify the *energy of a point* by the degree to which it violates the dispersion and boundary constraints. We define the energy of a dispersion point c_i as

$$E_D(c_i) = \sum_{j \neq i} \ell_{i,j}^2 + O_i \tag{4.1}$$

where $\ell_{i,j}$ is the dispersion penalty term and O_i captures the boundary violation term as in Equation 3.5. To propose new candidate positions for dispersion points, we consider *vacancy sites*, which are random locations inside the container not currently occupied by a dispersion point. A vacancy site c has energy

$$E_D(c) = \sum_{j=1}^{p} \ell_{c,j}^2 + O_c \tag{4.2}$$

where $\ell_{c,j}$ replaces c_i with c in Equation 3.1 and O_c replaces c_i with c in Equation 3.5. Vacancy sites with low energy represent positions that are more compatible with the current configuration, i.e., points where it would be the "easiest" to insert another ball.

A *neighbor* X_{nb} of a solution X can be found by *moving* a single dispersion point of X to a new location. The *neighborhood* $N(X)$ is the set of neighbors of X. To select the most promising neighbor of a given configuration X, we sample $N(X)$ by identifying the Q highest-energy dispersion points and the Q lowest-energy vacancy sites selected from a pool of $10p$ randomly generated candidates, where p is the total number of dispersion points. While Lai et al. [6] suggest generating only $5p$ vacancy sites, we increased this sample size to $10p$ to maintain sufficient search breadth, compensating for the lower sampling density inherent in three-dimensional space compared to two. We analyze the possible moves between these sets to identify the pairing that offers the best potential reduction in energy.

For a specific dispersion-vacancy move, the resulting configuration is merely a rough approximation of a feasible solution. Since vacancy sites are generated randomly, simply placing a point at one of these coordinates rarely resolves the feasibility-residual value, defined in Equation 3.6, to zero. Therefore, we do not evaluate the quality of this raw move directly. Instead, we pass this intermediate configuration to our local optimization subroutine, LocalOpt. LocalOpt serves as the mandatory bridge between the discrete combinatorial search and the continuous solution space. It utilizes a gradient-based solver, specifically the Limited-memory Broyden-Fletcher-Goldfarb-Shanno (L-BFGS) algorithm [17], to drive the rough candidate configuration toward the nearest smooth minimum of the energy function. Consequently, the Tabu Search algorithm never compares unrefined moves; it strictly compares the *locally optimized* outcomes of those moves to decide if a step is an improvement.

However, even with this precise local refinement, the global solution space of Equation 3.6 remains non-convex and populated with numerous local minima [6]. To prevent the algorithm from becoming trapped in the first deep valley it encounters, we integrate Monotonic Basin Hopping (MBH) as a heuristic for global exploration, as suggested by Lai et al. [6]. Unlike Tabu Search, which evolves a solution along a specific path, MBH samples the global landscape by "hopping" between independent basins (i.e., regions converging to a single minimum) [22]. It operates by taking a solution (X, D), performing random perturbations to jump into a neighboring basin, and then applying LocalOpt. We run ten such hops per call. Each hop displaces every coordinate by up to D, so the step scales with the problem rather than with the container, and its result is kept only if the residual does not rise: the walk is monotone, and the step never worsens the configuration it is given. A walk that also accepts uphill minima can cross barriers this one cannot; in our tests it reached the best basin no more often on the cube and less often on the star[p = 100, 4 seeds, l2], but Section V C shows one cell where crossing barriers matters.

MBH is applied inside the Tabu Search loop rather than as a separate outer layer, on the configuration that results from taking the chosen move (line 20 of Algorithm 1). The two heuristics address different failure modes and compose naturally there: the tabu memory prevents the search from re-taking a move it has just undone, while MBH perturbs whatever configuration that move produced. Placing it outside the loop would leave each tabu iteration to terminate in its own basin with no means of leaving it.

Algorithm 1 Tabu Search

Require: C (Container), X (configuration), $D \in \mathbb{R}_{>0}$, $\beta \in \mathbb{Z}_{>0}$, $Q \in \mathbb{Z}_{>0}$, $\epsilon \in \mathbb{R}_{>0}$

$Q \leftarrow \min\{Q, p\}$
$X \leftarrow \textsc{LocalOpt}(X, D)$
$X^{\text{best}} \leftarrow X$, $\mathit{NoImprove} \leftarrow 0$, $\mathit{TabuList} \leftarrow \{\}$
while $\mathit{NoImprove} \leq \beta$ and $E_D(X^{\text{best}}) > \epsilon$ **do**
  $p_1, \ldots, p_Q \leftarrow \textsc{HighEnergyPoints}(X, D, Q)$
  $q_1, \ldots, q_Q \leftarrow \textsc{VacancySites}(X, D, Q)$
  $X_{\text{nb}}^{\text{best}} \leftarrow X$, $e_{\text{nb}}^{\text{best}} \leftarrow \infty$, $\mathit{BestMove} \leftarrow \textsc{Nil}$
  for $(i, j) \in [Q] \times [Q]$ **do**
    $X_{\text{nb}} \leftarrow \textsc{LocalOpt}(\textsc{MovePoint}(X, p_i, q_j), D)$
    $a \leftarrow (\textsc{Move}(p_i, q_j)$ not forbidden$)$ **or** $(E_D(X_{\text{nb}}) < E_D(X^{\text{best}}))$
    if a **and** $E_D(X_{\text{nb}}) < e_{\text{nb}}^{\text{best}}$ **then**
      $X_{\text{nb}}^{\text{best}} \leftarrow X_{\text{nb}}$, $e_{\text{nb}}^{\text{best}} \leftarrow E_D(X_{\text{nb}})$
      $\mathit{BestMove} \leftarrow \textsc{Move}(p_i, q_j)$
    end if
  end for
  if $\mathit{BestMove} \neq \textsc{Nil}$ **then**
    $X \leftarrow X_{\text{nb}}^{\text{best}}$
    Add $\mathit{BestMove}$ and its reverse to $\mathit{TabuList}$
  end if
  $X \leftarrow \text{MBH}(X, D)$
  if $E_D(X) < E_D(X^{\text{best}})$ **then**
    $X^{\text{best}} \leftarrow X$
    $\mathit{NoImprove} \leftarrow 0$
  else
    $\mathit{NoImprove} \leftarrow \mathit{NoImprove} + 1$
  end if
end while
return X^{best}

Algorithm 1 outlines the Tabu Search procedure. To visualize the effect of Tabu Search, Fig. 6 illustrates a candidate move for a fixed radius D within a cubic container.

The dispersion point shown in red is in a high-energy state due to a boundary constraint violation (protrusion from the container). The yellow sphere represents a randomly generated vacancy site located in a feasible region. The red arrow indicates the proposed movement: displacing the violating particle to this vacancy site. In this specific example, because the target site satisfies all constraints (it is inside the boundary and non-overlapping), the feasibility-residual value is zero. Consequently, the subsequent LocalOpt routine will confirm this new configuration as stable with minimal energy. To prevent cycling, the algorithm explicitly forbids the reverse move in future iterations; the yellow dispersion point is temporarily restricted from returning to the high-energy neighborhood of the red sphere. This demonstrates the primary function of Tabu Search: executing coarse, global jumps to locate feasible regions, which are then refined by local optimization.

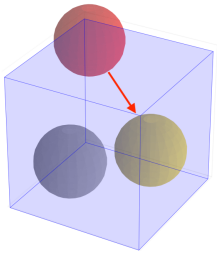


FIG. 6: **Visualization of a candidate neighbor generation.** The red sphere represents the point selected for movement (e.g., a high-energy point). The red arrow indicates the proposed displacement vector, targeting the location currently occupied by the yellow sphere.

B. Adjust Distance

After the Tabu Search procedure is completed, the resulting configuration X is a feasible solution, but typically not optimal. To find the nearest optimal solution $X^\star$ for the given configuration X, we employ the Sequential Unconstrained Minimization Technique (SUMT) [21] as proposed by Lai et al. [6] to convert the original CpDP (as a constrained optimization problem) into a sequence of unconstrained optimization problems and solve each until a feasible local minimum solution is found. Each unconstrained subproblem can be expressed as

$$\textit{Minimize} \quad \Phi_\mu(X, D) = -D^2 + \mu \times E(X, D) \tag{4.3}$$

where X is a candidate solution, D is the radius of the solution X, μ is the penalty factor to define the unconstrained optimization problem.

Algorithm 2 is the method to adjust a feasible configuration from Algorithm 1 into a local optimum. The distance adjustment method will iteratively minimize Equation 4.3 for strictly increasing values of μ. The $-D^2$ term in Equation 4.3 is to ensure that when we minimize this function, we promote maximizing D. Equation 3.5 contains $(\max\{0, D - d(c_i, v)\})^2$ which grows quadratically with D. The $-D^2$ term thus ensures Equation 4.3 is not dominated by $E(X, D)$ [6]. The distance adjustment procedure slightly adjusts the configuration X and the allowed minimum distance D through local optimization to find a local optimum of the original CpDP.

Algorithm 2 Adjust Distance

Require: X_0 (configuration), $D_0 \in \mathbb{R}_{>0}$, $K \in \mathbb{Z}_{>0}$, $\epsilon \in \mathbb{R}_{>0}$, $\mu_s \in \mathbb{R}_{>0}$ (default 1)

$X \leftarrow X_0$, $D \leftarrow D_0$, $\mu \leftarrow \mu_s$
for $i \leftarrow 1$ to K **do**
  $\mu \leftarrow 10 \cdot \mu$
  $(X, D) \leftarrow \textsc{LocalOptPhi}(X, D, \mu)$
  if (X, D) unchanged for three consecutive iterations **then**
    break
  end if
end for
if $E_D(X) \geq \epsilon$ **then** ▷ infeasible: snap to the exact bound
  $D \leftarrow \min\{0.95D,\ B(X)\}$
  $X \leftarrow \textsc{LocalOpt}(X, D)$
  $(X, D) \leftarrow \textsc{AdjustDistance}(X, D, 5, \epsilon, 10^3)$
end if
if $B(X) > 0$ **then**
  $D \leftarrow B(X)$ ▷ exactly what X supports
end if
return (X, D)

The LocalOptPhi function is a run of L-BFGS on the function Φ_μ from an initial solution (X, D) and penalty factor μ. We take $\mu_0 = 10$ and $\mu \leftarrow 10\mu$ with $K = 15$, so the schedule spans 10 to 10^{15}. A larger μ_0 makes the first gradient ill-conditioned: the penalty term then dominates $-D^2$ from the outset, and the first iterate is driven to satisfy the constraints at whatever radius it starts from rather than trading radius against violation. Starting small and multiplying is what lets each iteration begin near the previous minimiser.

Two quantities make the procedure self-correcting. Write

$$B(X) = \min\left\{ \min_{i<j} \tfrac{1}{2} d(c_i, c_j),\ \min_i d(c_i, \partial C) \right\}, \qquad (4.4)$$

the largest radius the configuration X admits. Both constraints tighten monotonically in D — constraint *(ii)* needs $d(c_i, c_j) \geq 2D$ and constraint *(iii)* needs $d(c_i, \partial C) \geq D$ — so the feasible radii for a fixed X are exactly the interval $[0, B(X)]$, and $B(X)$ is attained. Every D we report is therefore certified feasible by construction rather than by tolerance.

This matters at both ends of the loop. If SUMT terminates infeasible, D is snapped to $B(X)$ in one step instead of being walked down in fixed proportional decrements, which may exhaust its retries while still infeasible; the retry resumes the penalty schedule at $\mu = 10^4$ rather than restarting it, since the configuration it continues from is already close to feasible. And whichever side of $B(X)$ SUMT stops on, the radius returned is $B(X)$ itself. Above it, the tolerance $E < \epsilon$ alone would admit a violation of up to $\sqrt{\epsilon}$; below it, the radius would be conservative. In practice $B(X) = 0$ only when a point has left the container, and that case is reported as infeasible. (We return $B(X)(1 - 10^{-9})$, a margin against rounding in the final evaluation.) The lower guard is not cosmetic: $D = 0$ is a stationary point of Φ_μ at the lower bound whenever the points are well separated, since $E = 0$ and $\partial\Phi_\mu/\partial D = -2D$ both vanish there, so an over-estimated D_{init} can drive the procedure onto a trivially "feasible" answer — as it did for the H-box at $p = 1$, which returned $D = 0$ for every seed before the bound was imposed.

Note that LocalOpt is called in Algorithm 2 only on the infeasible branch, where D has just changed and minimizing $E_D(X)$ can therefore still help. It is not called in the main loop, where D is free and LocalOptPhi already varies it. Conversely it would not help to call LocalOptPhi in Algorithm 1, since Tabu Search is only designed for finding feasible configurations at a fixed D. It is important not to "rush" finding a maximal value of D, as finding a stable and feasible configuration X first often leads to a smoother maximization of D.

C. Global Optimization

The global optimization procedure is summarized in Algorithm 3. This algorithm represents a three-dimensional analogue to the TSGO framework established by Lai et al. [6]. While Lai et al. [6] utilized a time-bounded loop with continuous dependency on the global best state, our approach adopts an iterative multi-start strategy. This modification decouples the search trajectories, allowing each iteration to explore a distinct portion of the solution space independently before comparing against the current global optimum.

By shifting to an iteration-based structure, we ensure that the algorithm extensively samples the configuration space. Each iteration begins with a fresh randomization, preventing the solver from becoming trapped in a previously found local optimum. This is particularly crucial for non-convex containers where the feasible region may be disconnected.

Algorithm 3 Global Optimization

Require: $p \in \mathbb{N}$, C, $D_{\text{init}} \in \mathbb{R}_{>0}$, N (most restarts), R (patience), T (time limit), $\epsilon \in \mathbb{R}_{>0}$
Ensure: Best configuration (X^*, D^*)

$D \leftarrow D_{\text{init}}$
$X \leftarrow \textsc{RandomSolution}(p)$
$X \leftarrow \textsc{TabuSearch}(X, D)$
$X, D \leftarrow \textsc{AdjustDistance}(X, D, 15)$
$X^*, D^* \leftarrow X, D$
$s \leftarrow 0$
for $i = 1$ **to** N **do**
  if $s \geq R$ **or** more than T has elapsed **then**
    break
  end if
  $D_i \leftarrow D^*, \quad s \leftarrow s + 1$
  $X_i \leftarrow \textsc{RandomSolution}(p)$
  $X_i \leftarrow \textsc{TabuSearch}(X_i, D_i)$
  if $E_{D_i}(X_i) < \epsilon$ **then**
    $(X_i, D_i) \leftarrow \textsc{AdjustDistance}(X_i, D_i, 15)$
    if $D_i > D^*$ and $E_{D_i}(X_i) < \epsilon$ **then**
      if $D_i > (1 + 10^{-7})D^*$ **then**
        $s \leftarrow 0$
      end if
      $X^*, D^* \leftarrow X_i, D_i$
    end if
  end if
end for
$(X^*, D^*) \leftarrow \textsc{Polish}(X^*)$
return (X^*, D^*)

Within each iteration, the algorithm retains the rigorous local search operators utilized by Lai et al. [6]. An initial configuration is generated via RandomSolution, followed by TabuSearch to locate the nearest local minimum. We then employ a strict feasibility check: only if the feasibility-residual value $E_D(X)$ for the configuration X falls below the tolerance threshold ϵ does the algorithm proceed to AdjustDistance. This ensures that computational resources in the expensive refinement phase are reserved solely for valid, high-quality candidate solutions that have a genuine potential to improve upon the current global maximum D^*. At the local level, the optimization subroutines rely on a gradient-based solver governing Equation 3.6. This method exploits the specific geometry of the polyhedral container to achieve efficient convergence. Using gradient-based solvers over an almost-everywhere differentiable feasibility-residual function provides a smooth convergence path, offering greater stability than repulsion-based heuristics that rely on physical dynamical system simulations.

V. RESULTS AND EVALUATION

Experimentation is carried out on a personal computer with an Apple M2 Pro 3.5 GHz processor and 32GB RAM running on a 64-bit ARM-based MacOS 26.2[check hardware and OS line]. CPU times are given in seconds. The algorithm is implemented in Python 3.11, and the source code is publicly available (Source Code). The repository's `supplement` folder lists the centres of every configuration we report, at full double precision, with each container's faces and a standalone script, `verify_packing.py`, that needs only NumPy and none of our solver. From the centres alone it recomputes the largest feasible radius exactly, in any of the three norms: half the smallest pairwise distance, or the smallest distance from a centre to a face, whichever is less, with every centre inside the container. For all 377 configurations it agrees with the radius we report, which sits a relative 10^{-9} below it by design. Our specialized hybrid solver pairs Tabu Search with an L-BFGS optimizer to efficiently handle the problem's $3p+1$ variables and highly non-convex solution space, avoiding the local minima traps that stall standard generic solvers. We enforce a constraint violation tolerance of $\epsilon = 10^{-8}$ and truncate reported results to eight decimal places, ensuring the reported configurations are strictly feasible within the container geometry.

We evaluate our algorithm on five polyhedral containers of increasing geometric complexity: the unit cube, unit-length tetrahedron, H-shaped box, star-shaped polyhedron, and a ribbed ventilation cage. The boundaries for each are defined using a vectorized representation of inward-pointing normals for efficient distance calculations. To evaluate robustness, we benchmarked our algorithm against Packomania records [13], the results of Kazakov et al. [7], optimality certificates from Gurobi [23], and two general-purpose nonlinear programming solvers, IPOPT [24] and SLSQP [25, 26], run under an equal time budget. The protocol for that comparison (solvers, formulations, cells, budgets and statistics) was fixed in writing before any run, and is given in Section V G.

Every run is Algorithm 3: a first start from D_{init}, then restarts from fresh random configurations at the best radius found so far, until $R = \lceil 3 \ln p \rceil$ restarts in a row fail to raise it by more than a relative 10^{-7}, and at most $N = 40$ restarts and $T = 30$ minutes per run; a restart under way when the time runs out is finished. Tabu Search depth is $\beta = 10$ and the tabu-rearrangement parameter is $Q = \max(3, \text{round}(\ln p))$, which is the value $Q = 3$ of Lai et al. [6] for $p \leq 33$. Each cell (container, metric, p) is run from 20 independent seeds. The restarts are what put Tabu Search to work. From a start at or below $\hat{D}$, the first LocalOpt usually reaches $E_D < \epsilon$ on its own, and Tabu Search then performs no iterations at all; a restart at the best radius so far rarely does, so there the search has to relocate points to find a feasible configuration. In a pilot on twelve cells spanning all five containers and three metrics, with 25 starts per cell either way, four restarts per run found a larger radius than one start per run in six cells, by 1.0% to 6.7%[pilot from the old N = 4 design], was behind in none by more than 1.2×10^{-5} (relative), and used a third of the CPU time. How many restarts to make we settled by replaying 160 long runs (cube, tetrahedron, H-box and star; ℓ_2 and ℓ_1; $p = 10$ to 100; capped at 41

starts or 30 minutes) against stopping rules constant in p and growing as $\ln p$, $\sqrt{p}$ and p: $\lceil 3 \ln p \rceil$ used 81% of the capped CPU time, lost 0.003% of the radius on average, and lost more than 0.01% in one cell of 40. At large p the time limit binds first (Section V H). Q grows with p because, at a fixed budget, $Q = 5$ beat 3 and 6 on the cube at $p = 100$, and $Q = 4$ beat 3 in 12 of 24 paired runs, and lost in 5, on the tetrahedron and H-box at $p = 60$.

POLISH tightens the run's best configuration by sequential linear programming. Every distance in the constraints is a convex function of the points, so its linearisation at the current configuration is a lower bound on it, and a configuration that satisfies the linearised constraints satisfies the original ones. Each step solves that linear program, for the points and the radius, inside a trust region; its result is certified exactly and kept only if the radius rises, so the polish never lowers it. It closes the gap of a few parts in 10^5 by which the penalty method stops short of a configuration's local maximum, and with it the cube at $p = 100$ reaches the Packomania record. These values are recorded in every row of our results table.

a. Escape phase. A restart runs Tabu Search at the best radius found so far, and Tabu Search moves points only while the residual is positive. A random start that is already feasible at that radius is therefore returned unmoved, however poor its layout, and a run whose best configuration is held in place by a narrow feature of the container can repeat it on every restart. To test whether this explains failures we had observed, we wrote an *escape phase*: once the restarts stop, further restarts run Tabu Search at a target 2% above the best radius, where such a start is infeasible and has to move. Each grows its radius from wherever Tabu Search leaves it and replaces the best only if it beats it, so the phase can only raise a run's result. It stops after as many restarts without a gain as the main loop allows, or after 15 minutes; these are the only settings we tried. A saved run can be continued with it, drawing the random streams the run itself would have drawn, so a continuation is the run the option would have produced.

The escape phase is a diagnosis, not part of the method. We wrote it after seeing runs stall, and applied it afterwards only to the three cells in which we had seen runs held in place this way: example 6 of Stoyan and Yaskov [8], the H-box under ℓ_∞ at $p = 56$, and the ribbed cage under ℓ_2 at $p = 100$. Every table, and every figure other than those of these three cells, reports the defaults. We have not evaluated the phase across the benchmark, and do not claim it would help elsewhere.

The initial radius is estimated by filling a fraction $\rho_{\text{init}} = 0.25$ of the container with p balls [6, 27], then capping at the container's inradius:

$$\hat{D} = \min\left\{ \sqrt[3]{\frac{\rho_{\text{init}}\, V_{\text{container}}}{\kappa_d\, p}},\ r_{\text{in}}(C) \right\}. \qquad (5.1)$$

Here κ_d is the volume of the unit ball of the metric d, so that a ball of radius D occupies $\kappa_d D^3$: it is $\frac{4}{3}\pi$ for ℓ_2, 8 for ℓ_∞, whose ball is a cube of side $2D$, and $\frac{4}{3}$ for ℓ_1, whose ball is a cross-polytope. The metric belongs in the estimate because the heuristic is a statement about volume, and the three balls differ in volume by a factor of six at the same radius. Using the Euclidean constant throughout, as is tempting, over-estimates $\hat{D}$ by 1.24× under the sup norm — with consequences described below. The cap is not a refinement but a necessity. A volume-based estimate knows nothing about shape, and on containers whose volume is distributed into thin or branching regions it over-estimates badly — by 1.5× on the H-box and 2.1× on the cage. Since no D above the inradius is feasible at any p, an uncapped estimate starts the search outside the feasible range of radii entirely, which is the regime in which $D = 0$ becomes an attracting stationary point of Φ_μ; together with the bound $B(X)$ of Equation 4.4 this is what removed the $D = 0$ collapse at $p = 1$. The same expression is used by both the solver and the experiment driver, so the two cannot disagree.

$\hat{D}$ is an estimate, and neither direction of error is free. Starting above the achievable radius sends Algorithm 2 into its infeasible branch: D is snapped to $B(X)$ for whatever configuration it is then holding, and the search continues from that configuration rather than from a fresh one. On the unit cube at $p = 2$ under the sup norm this is the difference between 0.2299 and 0.2500, the latter being exactly optimal — two cubes of side $\frac{1}{2}$ tile the unit cube. Starting below it is not harmless either. Φ_μ drives D upward, but Tabu Search has by then settled the points into an arrangement sized for the smaller radius, and that arrangement can confine the climb. On the unit cube at $p = 20$ under the sup norm, all four seeds started at $1.30\hat{D}$ reach the optimum $\frac{1}{6}$, a $3 \times 3 \times 3$ arrangement with points to spare, while five of the twelve started at or below $\hat{D}$ end at $D = \frac{1}{8}$, the $4 \times 4 \times 4$ one.

We therefore treat the initial radius as a second multi-start dimension. A cell's seeds are distributed over a geometric ladder of K rungs spanning a factor λ either side of the estimate,

$$D_{\text{init}}^{(k)} = \lambda^{\frac{2k}{K-1}-1}\, \hat{D}, \qquad k = 0, \ldots, K-1, \qquad (5.2)$$

each clipped at $r_{\text{in}}(C)$, with $K = 5$ and $\lambda = 1.7$. Seed s starts on rung $s \bmod K$, so each rung receives four of a cell's twenty seeds. The spread is set by measurement rather than tuned: across the non-Euclidean cells the best radius found lies between $0.77\hat{D}$ and $1.59\hat{D}$, so $\lambda = 1.7$ brackets the observed range with margin. Since every reported radius is already the best over a cell's seeds, this costs no additional runs — it widens the multi-start from configurations alone to pairs of configuration and initial radius. The inradius used for the cap is computed numerically, by maximising clearance from the best of 64 sampled interior points, and so can only err low; it is a conservative cap.

A start high on the ladder can end infeasible, and such a run is not discarded. It is restarted from the same seed

at a lower radius, half a rung at a time: a run that fails from $D_{\text{init}}^{(k)}$ is restarted from $\frac{1}{2}\left(D_{\text{init}}^{(k-1)} + D_{\text{init}}^{(k)}\right)$, one that fails from there is restarted from $D_{\text{init}}^{(k-1)}$, and so on down to $D_{\text{init}}^{(0)}$. Only a run still infeasible from $D_{\text{init}}^{(0)}$ is reported infeasible. Because a restart keeps its seed, it begins from the same random configuration and differs from the failed attempt only in its radius; steps that the inradius cap merges are taken once, since repeating a radius would repeat the failure. Every reported time includes all of a run's attempts. In the sweep reported here the fallback was needed by 46 of 4,800 runs: 43 succeeded half a rung down and three after two steps, and no run ended infeasible.

The ladder is not arbitrary: the optimum is bracketed by two bounds that hold for any norm and any container, convex or not. Write $V = \text{vol}(C)$ and

$$C_{-D} = \{x \in C : d(x, \partial C) \geq D\} \tag{5.3}$$

for the set of admissible centres at radius D.

Lemma 3. *Let $D^\star(p)$ be the optimal radius for p points in C under a norm whose unit ball has volume κ_d. Then*

$$\underline{D}(p) \;\leq\; D^\star(p) \;\leq\; \min\left\{\left(\frac{V}{\kappa_d\, p}\right)^{1/3},\; r_{in}(C)\right\},$$

where $\underline{D}(p) = \sup\left\{D > 0 : \text{vol}(C_{-D}) > 8\kappa_d(p-1)D^3\right\}$.

Proof. The balls of radius D about the points of any feasible configuration lie in C, by constraint (*iii*), and have pairwise disjoint interiors, since $d(c_i, c_j) \geq 2D$ and the triangle inequality forbids a common interior point. Their volumes therefore sum to at most V, so $p\kappa_d D^3 \leq V$. And $D \leq d(c_1, \partial C) \leq r_{\text{in}}(C)$. Together these give the upper bound.

For the lower bound, fix D with $\text{vol}(C_{-D}) > 8\kappa_d(p-1)D^3$. By the argument just given, any subset of C_{-D} whose points are pairwise at least $2D$ apart has at most $V/(\kappa_d D^3)$ elements, so there is such a subset Y of largest size. Every $x \in C_{-D}$ lies within distance $2D$ of some $y \in Y$, as otherwise $Y \cup \{x\}$ would be larger. Hence C_{-D} is covered by the balls of radius $2D$ about the points of Y, and $\text{vol}(C_{-D}) \leq |Y|\,\kappa_d(2D)^3$. By the choice of D this forces $|Y| > p-1$. Any p points of Y are then a feasible configuration at radius D: they lie in C_{-D}, which is constraint (*iii*), and are pairwise $2D$ apart, which is constraint (*ii*). So $D^\star(p) \geq D$, and taking the supremum over such D gives the lower bound. □

Nothing in the proof uses convexity, the shape of the ball beyond its volume, or the dimension beyond the exponent. With the cap inactive the upper bound is $\rho_{\text{init}}^{-1/3}\hat{D} = 1.587\hat{D}$, so the top rung, at $1.7\hat{D}$, starts above the optimum in every cell: it is on the fallback that it depends. On the two convex containers, where C_{-D} is an intersection of half-spaces and its volume is exact, $\underline{D}(p)$ lies between $0.51\hat{D}$ and $0.70\hat{D}$ for $2 \leq p \leq 84$ in all three metrics, against best radii found between $0.80\hat{D}$ and $1.59\hat{D}$.[lower bound not recomputed this round] The top of that range is the upper bound itself: under the sup norm, eight cubes of side $\frac{1}{2}$ tile the unit cube, and the $p = 8$ radius $\frac{1}{4}$ is exactly $\rho_{\text{init}}^{-1/3}\hat{D}$. The lower bound is what the covering argument can certify, not what a search reaches: a start at $\underline{D}(p)$ is guaranteed to admit a feasible configuration, but lies at or below the bottom rung, where the sup-norm optima of the previous paragraph are missed. On the non-convex containers the bound needs a lower estimate of $\text{vol}(C_{-D})$, which a decomposition into convex pieces with disjoint interiors supplies, since their inner bodies are disjoint subsets of C_{-D}; we have not evaluated it there.

The effect is confined to the non-Euclidean metrics, and is not small. Under the sup norm the uncorrected estimate produced radii that violated monotonicity in p — D rising as points were added, which is impossible, since a configuration of $p+1$ points contains one of p — in nineteen cells, every one of them sup norm and none of them ℓ_1 [from the pre-wipe experiment], exactly the asymmetry the constants κ_d predict.

A. Cube

We benchmark our algorithm in the unit cube against analytical optima for $p \leq 10$ [12] and best-known configurations for $p > 10$ from Packomania [13]. We measure performance using absolute error ($A_{err} = D_{\text{Ours}} - D_{\text{Best}}$) and relative error ($R_{err} = A_{err}/D_{\text{Best}} \times 100\%$).

Table I shows our radius matching the published value at every $p \leq 40$, the proven optima for $p \leq 10$ included, to within 2.3×10^{-8} (relative), which is the rounding of the published values. At $p = 50$ and $p = 64$ it is short by 8.6×10^{-7} and 4.1×10^{-6}, and at $p = 100$ it matches the record. Fig. 9 shows the same comparison across the whole range. The comparison with general-purpose solvers is in Section V G.

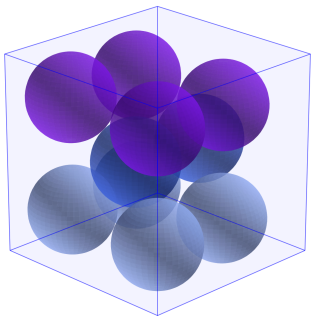


FIG. 7: Best-achieved 10-point configuration for the p-dispersion problem in the unit cube using the Euclidean metric ($D = 0.21428571$).

TABLE I: Cube: best radius found in each metric, truncated to eight decimals. For ℓ_2 the published value (Packomania) and the relative difference $R_{\mathrm{err}} = (D - D_{\mathrm{ref}})/D_{\mathrm{ref}}$ are given; negative values are cells where the published value is ahead. Packomania marks N = 1 and 3-10 as proven optimal (in bold on its page); the rest are best known.

p	D, ℓ_2	Packomania	R_{err} (%)	D, ℓ_1	D, ℓ_∞
1	0.49999999	0.50000000	−0.0000	0.49999999	0.49999999
2	0.31698729	0.31698730	−0.0000	0.37499999	0.24999999
3	0.29289321	0.29289322	−0.0000	0.33333333	0.24999999
4	0.29289321	0.29289322	−0.0000	0.33333333	0.24999999
5	0.26393202	0.26393202	+0.0000	0.29999999	0.24999999
6	0.25735931	0.25735931	+0.0000	0.29999999	0.24999999
7	0.25013615	0.25013615	+0.0000	0.28571428	0.24999999
8	0.24999999	0.25000000	−0.0000	0.27272727	0.24999999
9	0.23205080	0.23205081	−0.0000	0.26923076	0.16666666
10	0.21428571	0.21428571	+0.0000	0.26086956	0.16666666
20	0.17840719	0.17840720	−0.0000	0.22222222	0.16666666
27	0.16666666	0.16666667	−0.0000	0.20689655	0.16666666
30	0.16018862	0.16018862	+0.0000	0.19999999	0.12499999
40	0.14705882	0.14705882	+0.0000	0.18749999	0.12499999
50	0.13595439	0.13595451	−0.0001	0.17647058	0.12499999
64	0.12674302	0.12674355	−0.0004	0.16666666	0.12499999
100	0.11138234	0.11138235	−0.0000	0.14574898	0.09999999

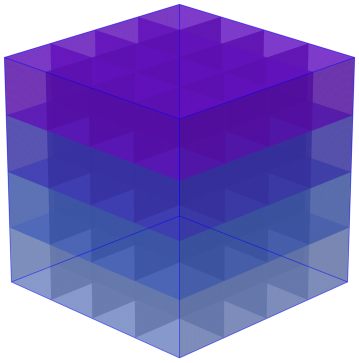


FIG. 8: Best configuration of $p = 64$ points in the unit cube under the ℓ_∞ metric ($D = 0.12499999$): 64 cubes of side $\frac{1}{4}$ in a $4 \times 4 \times 4$ grid, the optimum $D = \frac{1}{8}$. Colour gives height.

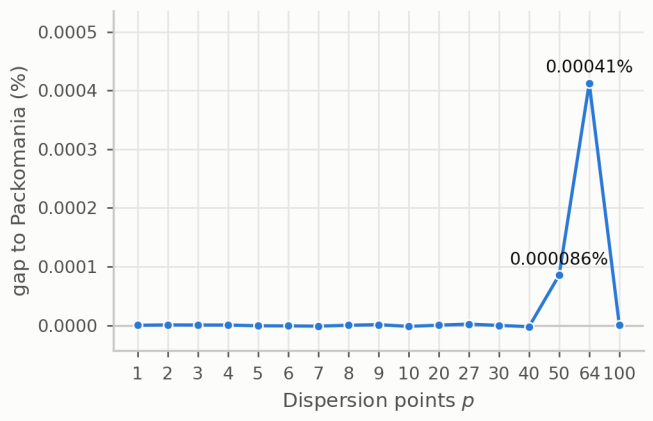


FIG. 9: Unit cube, Euclidean metric: gap between the Packomania record D_{P} and our best radius D, $(D_{\mathrm{P}} - D)/D_{\mathrm{P}}$ in percent. Zero means our radius matches the record; only $p = 50$ and $p = 64$ show a gap, below 0.0005%.

Fig. 7 shows the best Euclidean configuration for ten points. Under the ℓ_∞ metric each ball is an axis-aligned cube of side $2D$, and at $p = k^3$ the unit cube admits an exact tiling: k^3 cubes of side $\frac{1}{k}$ fill it, so $D = \frac{1}{2k}$ meets the upper bound of Lemma 3, $(V/(\kappa_\infty p))^{1/3} = (1/(8k^3))^{1/3} = \frac{1}{2k}$, and is optimal. At $p = 27$ our algorithm finds the $3 \times 3 \times 3$ grid in eight of twenty runs, at $\frac{1}{6}$ to within the relative 10^{-9} by which every reported radius is shrunk; the rest settle on $D = \frac{1}{8}$. At $p = 64$ the tiling is harder to find: two of twenty runs reach the $4 \times 4 \times 4$ grid (Fig. 8), again to within 10^{-9}, and the other eighteen settle on $D = \frac{1}{10}$.

B. Tetrahedron

We benchmark our algorithm against the best-known results for p-dispersion in the unit tetrahedron reported by Kazakov et al. [7] for $p = 1, \ldots, 10$. Additionally, we report on our best-achieved minimum distances for $p = 20, 35, 56, 84$, the subsequent tetrahedral numbers. Kazakov et al. [7] mention that for three-dimensional space, the computational complexity of their algorithm increases significantly; hence they did not report on instances for $p > 10$. We execute our global optimization

algorithm using the same parameters as in previous sections the packing density-based initialization ($\rho_{\text{init}} = 0.25$) for higher values of p. While Kazakov et al. [7] report results up to five decimal places, we report our results truncated to eight decimal places to maintain consistency with other sections of this paper.

Table II presents the absolute difference $A_{\text{err}} = D_{\text{Ours}} - D_{\text{Benchmark}}$ and the relative percentage improvement R_{err} between our solution and the benchmark. As shown in the table, our algorithm consistently identifies packing configurations with equal or strictly larger minimum pairwise distances than Kazakov et al. [7].

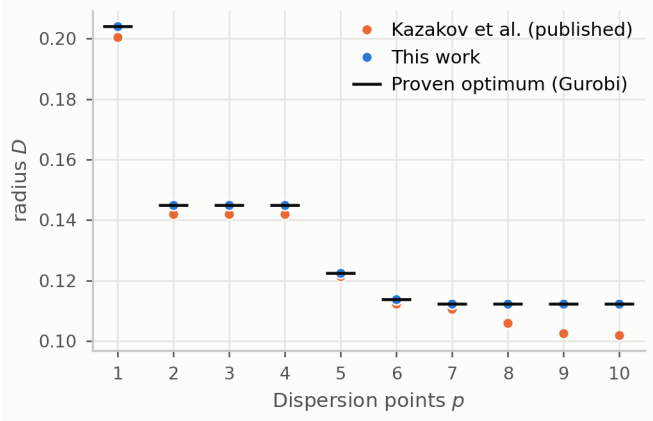
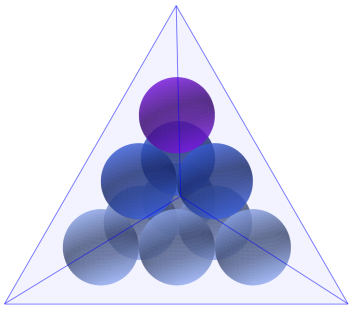


FIG. 10: Unit tetrahedron, Euclidean metric: the radius reported by Kazakov et al. [7] (orange), our best radius (blue), and the optimum Gurobi proves (black dash). At every p our radius sits on the proven optimum; the benchmark falls below it by 0.8% to 9.2%.

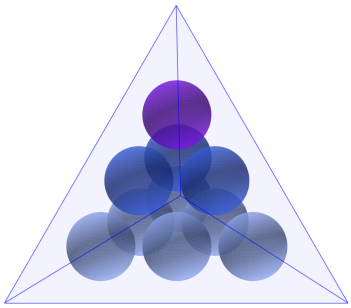
To further analyze these improvements, Fig. 10 illustrates the comparison between our algorithm and the results of Kazakov et al. [7]: our radius is at or above the published benchmark at every p it reports. Gurobi proves that the optimum at $p = 10$ is 0.112372 (Section V G), which our radius matches and the benchmark misses by 9.2%.

The size of the margin over Kazakov et al. [7] deserves a caveat. At $p = 1$ the answer is exactly the inradius of the unit regular tetrahedron, $1/(2\sqrt{6}) \approx 0.20412$, which both methods here recover to eight decimal places and which the published benchmark misses by 1.8%. A gap against a benchmark that is itself 1.8% short at the one p where the optimum is elementary should be read as a statement about the benchmark's tolerance as much as about either solver. Fig. 11 provides a visual comparison of the best configuration found by our algorithm against the corresponding configuration derived from the parameters in Kazakov et al. [7] for $p = 10$, a tetrahedral number, where the configuration settles into the 1–3–6 triangular arrangement the container's symmetry admits.

We achieve a minimum improvement of approximately 0.8% for all tested values of p compared to Kazakov et al. [7], with the performance gap widening to over 10% as p reaches the limits of the previously reported benchmarks.



(a) Our best-achieved configuration for $p = 10$ ($D = 0.11237243$).



(b) The same configuration drawn at the radius ($D = 0.10200$) reported for $p = 10$ by Kazakov et al. [7].

FIG. 11: Visual comparison of packing density for 10 dispersion points in the unit tetrahedron, an improvement of 10.2% over the benchmark radius. Both panels show the same configuration from the same viewpoint and at the same scale, so the radius is the only difference between them: our method (a) leaves the balls in contact, while at the benchmark radius (b) visible slack remains.

Under the other two metrics the balls change shape: under ℓ_1 each is an octahedron, under ℓ_∞ an axis-aligned cube. Fig. 12 shows the best configurations found for $p = 35$, a tetrahedral number, under all three metrics. Each builds up in layers from the base of the tetrahedron, the spheres into a close-packed pile, the octahedra into a pointed pyramid and the cubes into stepped tiers, as the slanted walls force each layer to be narrower than the one below.

Under ℓ_2 the layers form the tetrahedral pile. For $p = n(n+1)(n+2)/6$ spheres stacked in n layers, with

TABLE II: Tetrahedron: best radius found in each metric, truncated to eight decimals. For ℓ_2 the published value (Benchmark (Kazakov)) and the relative difference $R_{\text{err}} = (D - D_{\text{ref}})/D_{\text{ref}}$ are given; negative values are cells where the published value is ahead. Not certified optima. At p = 1 the exact answer is the inradius of the unit regular tetrahedron, $1/(2 \cdot \sqrt{6}) = 0.20412415$, which this benchmark misses by 1.8%; the reported gap to it should be read with that in mind.

p	D, ℓ_2	Benchmark (Kazakov)	R_{err} (%)	D, ℓ_1	D, ℓ_∞
1	0.20412414	0.20045000	+1.8329	0.22833941	0.14795327
2	0.14494897	0.14204000	+2.0480	0.17776456	0.11416970
3	0.14494897	0.14204000	+2.0480	0.17125653	0.11027435
4	0.14494897	0.14204000	+2.0480	0.16406441	0.10859672
5	0.12247448	0.12147000	+0.8269	0.15105889	0.09294638
6	0.11387082	0.11243000	+1.2815	0.14553094	0.09034817
7	0.11237243	0.11062000	+1.5842	0.14113995	0.08921895
8	0.11237243	0.10600000	+6.0117	0.13905945	0.08789128
9	0.11237243	0.10250000	+9.6316	0.13584155	0.08682227
10	0.11237243	0.10200000	+10.1690	0.13313203	0.08577896
20	0.09175170	–	–	0.11176187	0.07088498
30	0.07752551	–	–	0.10094528	0.06251369
35	0.07752551	–	–	0.09706067	0.06039796
40	0.07026264	–	–	0.09352178	0.05708485
50	0.06711869	–	–	0.08824806	0.05429836
56	0.06711869	–	–	0.08546645	0.05300470
84	0.05917517	–	–	0.07632857	0.04722404
100	0.05330231	–	–	0.07256921	0.04427507

neighbours touching and the outer spheres touching the walls, the centres fill a tetrahedron of edge $2(n-1)D$ inset by D from every face, whose edge is $1 - 2\sqrt{6}\,D$. Equating the two gives

$$D_n = \frac{1}{2\,(n - 1 + \sqrt{6})}. \tag{5.4}$$

Our radius matches D_n at every tetrahedral number in our grid, $p = 4, 10, 20, 35, 56, 84$ $(n = 2, \ldots, 7)$, to within the relative 10^{-9} by which every reported radius is shrunk. At $n = 3$, $D_3 = 1/(2(2+\sqrt{6})) \approx 0.112372$ is the optimum Gurobi certifies at $p = 10$; for $n \geq 4$ the pile is not known to be optimal, and we report the match, not optimality.

C. H-Box

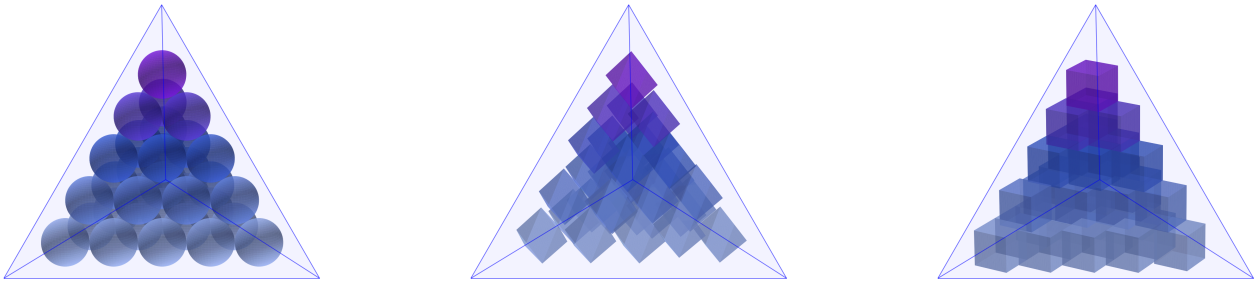
Non-convex containers, such as the H-shaped polyhedron, present significant geometric challenges for the CpDP. Unlike convex domains, non-convex shapes often feature disconnected feasible regions and obstructed lines of sight, where the straight-line segment connecting two valid points may pass through the infeasible exterior [5]. These characteristics create numerous "trapped" local minima that can stall standard optimization algorithms [6]. The H-shaped polyhedron was created with inspiration from its two-dimensional analogue: the H-shaped polygon [5].

Table III reports the best radius found for each p in all three metrics. No published values exist for this container; the comparison with general-purpose solvers is in Section V G, where the H-box is one of the containers on which they fall behind.

TABLE III: H-box: best radius found in each metric, truncated to eight decimals. No published values exist for this container.

p	D, ℓ_2	D, ℓ_1	D, ℓ_∞
1	0.49999999	0.49999999	0.49999999
2	0.49999999	0.49999999	0.49999999
3	0.49999999	0.49999999	0.49999999
4	0.49999999	0.49999999	0.49999999
5	0.49999999	0.49999999	0.49999999
6	0.49999999	0.49999999	0.49999999
7	0.49999999	0.49999999	0.49999999
8	0.41093596	0.46874999	0.37499999
9	0.39725848	0.46153846	0.37499999
10	0.39360891	0.45833333	0.37499999
20	0.31141847	0.38095238	0.24999999
30	0.29291737	0.34862385	0.24999999
40	0.27390743	0.33333333	0.24999999
50	0.25588218	0.31249999	0.24999999
100	0.20710694	0.26016260	0.16666666

Under the sup norm each ball is an axis-aligned cube of side $2D$, and at $p = 56$ the H-box admits an exact tiling: 56 cubes of side $\frac{1}{2}$ have total volume $56 \cdot \frac{1}{8} = 7$, the volume of the container. The radius $D = \frac{1}{4}$ therefore meets the upper bound of Lemma 3, $(V/(\kappa_\infty p))^{1/3} = (7/(8 \cdot 56))^{1/3} = \frac{1}{4}$, and is optimal. Our algorithm with



(a) ℓ_2: spheres ($D = 0.07752551$). (b) ℓ_1: octahedra ($D = 0.09706067$). (c) ℓ_∞: cubes ($D = 0.06039796$).

FIG. 12: Best configurations of $p = 35$ points in the unit tetrahedron under the ℓ_2, ℓ_1 and ℓ_∞ metrics. Under ℓ_2 the spheres form the five-layer tetrahedral pile of Eq. 5.4. Colour gives height.

its defaults does not find it: of sixty runs, twenty-four settle on $D = \frac{3}{14}$ and thirty-six on $D = \frac{1}{6}$, at 86% and 67% of the optimum. The tiling needs every cube on the grid at once. The monotone basin hopping inside Tabu Search cannot make that rearrangement, and a restart at $\frac{3}{14}$ or $\frac{1}{6}$ is feasible at once, so Tabu Search leaves it unmoved. With a basin-hopping walk that also accepts uphill minima, one run of ten finds the tiling, and the same run with ours does not.[seed 9; legacy MBH vs current, all else equal] Continued with the escape phase (Section V 0 a), all twenty of the first twenty runs reach the tiling, at $\frac{1}{4}$ to within the relative 10^{-9} by which every reported radius is shrunk (Fig. 13). Table III reports the defaults; this cell is outside its grid.

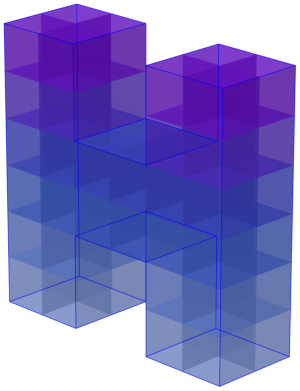


FIG. 13: The optimal configuration of $p = 56$ points in the H-shaped container under the ℓ_∞ metric, $D = \frac{1}{4}$ (0.24999999): 56 cubes of side $\frac{1}{2}$ tile the container exactly. Found by the escape phase in all 20 runs continued with it, and by none of 60 runs with the defaults. Colour gives height, so each layer of the tiling shows as a band.

D. Star-Shaped Polyhedron

The star-shaped polyhedron draws inspiration from the Kepler-Poinsot polyhedron [28] and serves as a rigorous test case for non-convex optimization due to its distinctive central convex core connected to six narrow, tapering spikes. This originally developed geometry creates significant "bottlenecks" between the feasible sub-regions, making it difficult for optimization algorithms to migrate points from one spike to another. Unlike convex containers where local descent directions are often globally informative, the star-shaped polyhedron requires an algorithm capable of traversing infeasible space to escape the local optima found within individual spikes.

In Table IV, we report the best minimum distances (D_{Ours}) achieved by our algorithm for a range of p values, up to $p = 100$.

The results indicate that our algorithm effectively exploits the available volume even as the number of points increases. For $p = 1$, the algorithm identifies the globally optimal placement within the central core. As p increases to 6, the algorithm successfully distributes points into the six available spikes, maintaining a high separation distance of $D \approx 0.9030$. From $p = 7$ on, crowding effects force a gradual reduction in the minimum distance D as the central core and spikes become filled.

Fig. 14 shows the best configuration found for $p = 100$. Only 24 of the 100 balls sit in the central core; the other 76 are split almost evenly among the six spikes, 12 or 13 in each, and the outermost centres lie 3.5 of the 5 units from the centre of the star to a tip. The algorithm does not crowd the core and leave the spikes empty: it carries points through the bottlenecks and fills every spike to a similar depth.

TABLE IV: Star: best radius found in each metric, truncated to eight decimals. No published values exist for this container.

p	D, ℓ_2	D, ℓ_1	D, ℓ_∞
1	1.41421356	1.99999999	0.99999999
2	0.97597050	1.19999999	0.83333333
3	0.90296402	1.06666666	0.73529411
4	0.90296402	1.01960784	0.71917808
5	0.90296402	0.99999999	0.71428571
6	0.90296402	0.99999999	0.71428571
7	0.81657908	0.95238095	0.71428571
8	0.76614288	0.93333333	0.62499999
9	0.72337218	0.92307692	0.59523809
10	0.70710678	0.92307692	0.58988763
20	0.58567331	0.74999999	0.49999999
30	0.51339550	0.66600480	0.42372881
40	0.47557671	0.61931435	0.41666666
50	0.44641227	0.58655215	0.38461538
100	0.36219729	0.48205331	0.30660377

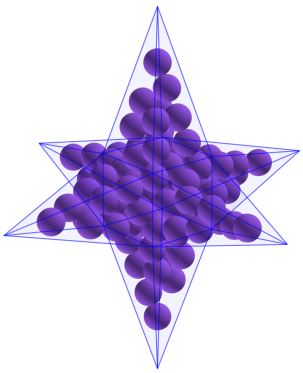


FIG. 14: Best achieved configuration of $p = 100$ points in the star-shaped container under the Euclidean metric ($D = 0.36219729$): 24 balls in the central core and 12 or 13 in each of the six spikes.

E. Ribbed Ventilation Cage

Finally, we apply our algorithm to the "ribbed ventilation cage," a highly complex non-convex polyhedron designed to model modern industrial computer casings and heat sinks. Unlike the H-shaped box or Star-shaped polyhedron, which feature large distinct sub-regions, this container is characterized by numerous corridors and separated legs. The inclusion of numerous internal pillars and ribs creates a "maze-like" feasible region, introducing frequent discontinuities that pose severe challenges for traditional gradient-based nonlinear optimizers.

TABLE V: Cage: best radius found in each metric, truncated to eight decimals. No published values exist for this container.

p	D, ℓ_2	D, ℓ_1	D, ℓ_∞
1	1.58113882	1.99999999	1.49999999
2	1.58113882	1.99999999	1.49999999
3	1.58113882	1.99999999	1.49999999
4	1.54166666	1.83333333	1.49999999
5	1.54166666	1.83333333	1.49999999
6	1.54166666	1.74999999	1.49999999
7	1.49999999	1.64285714	1.24999999
8	1.49999999	1.58333333	1.24999999
9	1.45668433	1.58333333	1.24999999
10	1.45668433	1.49999999	1.24999999
20	1.12942051	1.42105263	0.99999999
30	0.99999999	1.30952380	0.83333333
40	0.94245771	1.19999999	0.74999999
50	0.88970556	1.14285714	0.74999999
100	0.74175552	0.95109780	0.66666666

The particular design of this ribbed ventilation cage polyhedral container is original, although the complexity is practically motivated. In thermal engineering, components utilize ribbed structures, fins, and microstructures to maximize surface area and improve heat dissipation capabilities [29]. Solving the CpDP on such a geometry is therefore relevant for optimizing the layout of sensors, cooling pipes, or modular components within densely packed electronic enclosures. We evaluate our algorithm using both the Euclidean metric (ℓ_2), representing standard spherical component packing, and the sup-norm (ℓ_∞), which is critical for modeling the placement of cubic or box-shaped components often found in hardware design.

Table V reports the best radius found for each p in all three metrics.

Fig. 15 and Fig. 16 show the best configurations found for $p = 100$ points in this container under the Euclidean and ℓ_∞ metrics.

The cage has a trap of its own. Of the 900 cage runs, 75 stop at exactly $D = \frac{1}{2}$, and at $p = 100$ under ℓ_2 sixteen of twenty do, so that cell's best is a best of four. Continued with the escape phase (Section V 0 a), all sixteen leave it, reaching 0.726 to 0.742, and the best radius of the cell rises from 0.74176 to 0.74220. Every continuation used its full 15 minutes, a few restarts at this size, so this cell is limited by time rather than by the trap.[old run reached 0.74433]

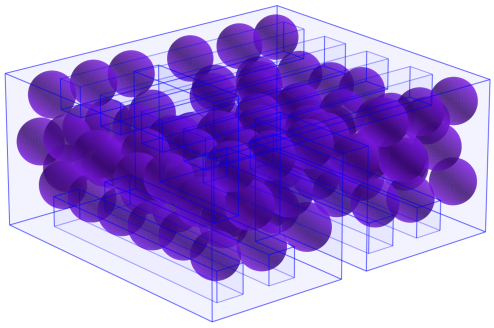


FIG. 15: Best achieved configuration for $p = 100$ points using the Euclidean metric ($D = 0.74175552$).

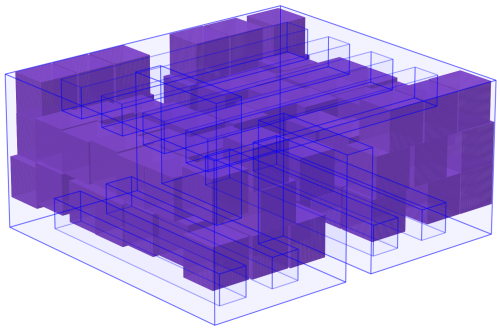


FIG. 16: Best achieved configuration for $p = 100$ points using the ℓ_∞ metric ($D = 0.66666666$).

The successful dispersion of $p = 100$ points in this environment demonstrates the robustness of our approach. Despite the numerous internal obstacles that fracture the search space, the algorithm achieves a uniform distribution, effectively utilizing the narrow corridors between pillars. This result validates the algorithm's capability to handle large-scale dispersion problems ($p = 100$) within containers that exhibit both macro-scale non-convexity and micro-scale geometric obstructions.

TABLE VI: Spheres of radius one packed into the eight domains of Stoyan and Yaskov [8]: their count, ours, our best radius at their count, and the time at which one of our runs first held radius one at their count, against their reported time.

	Spheres			Time (min)	
Example	Theirs	Ours	D at theirs	Ours	Theirs
1	93	97	1.0068	6.8	41
2	91	93	1.0069	2.3	34
3	86	88	1.0069	1.6	47
4	81	82	1.0065	1.9	54
5	79	79	1.0045	3.5	60
6	30	31	1.0072	0.9	2
7	58	59	1.0059	0.9	7
8	73	80	1.0326	0.2	12

F. Comparison with Stoyan and Yaskov

Stoyan and Yaskov [8] pack congruent spheres of radius one into eight multi-connected polyhedral domains: a truncated box, or a cuboid, from which through-prisms, polyhedral cones and dihedral angles are removed, so that the domain has holes. For each they report the largest number of spheres they packed and the time it took. Packing p spheres of radius one is the decision version of our problem under ℓ_2: they fit exactly when $D^\star(p) \geq 1$. We rebuilt the eight domains from the coordinates given in their paper and ran our algorithm with the settings of the main experiment, 10 seeds per count, at their count and the four counts above it, and further in the two domains where every one of those counts fitted. A count is fitted when some seed returns a certified radius of at least one.

We fit more spheres than published in seven of the eight domains, by one to seven, and as many in the fifth (Table VI, Fig. 18). At their published counts, 71 of our 80 runs reach radius one, and the first does so within 7 minutes in every domain. Their times were measured on an AMD Athlon 64 X2 6000+ in Delphi, so the time columns indicate scale rather than a like-for-like comparison. Fig. 17 shows each domain at our count. The gains are largest in the box of example 1, which has no holes, and in example 8, where the hole is shortest.

The domain of example 6 shows how the search can stall. The cuboid there is pierced by a tapered hole whose floor leaves a wall below it, 1.97 thick at one end and 1 at the other. A sphere that ends in that wall, against the end face, the bottom and the sloping floor of the hole, can have radius at most 0.93364, the inradius of that wedge. Eight of the ten seeds at every count stop at exactly this radius, because a restart at the best radius found so far is already feasible there and Tabu Search, which runs only until the target radius is feasible, never has to move the trapped sphere. The two seeds that avoid the wall fit 31 spheres.

To see whether the trap or the domain limits the count, we continued the 50 saved runs of example 6 with the escape phase (Section V 0 a), at whose target a sphere

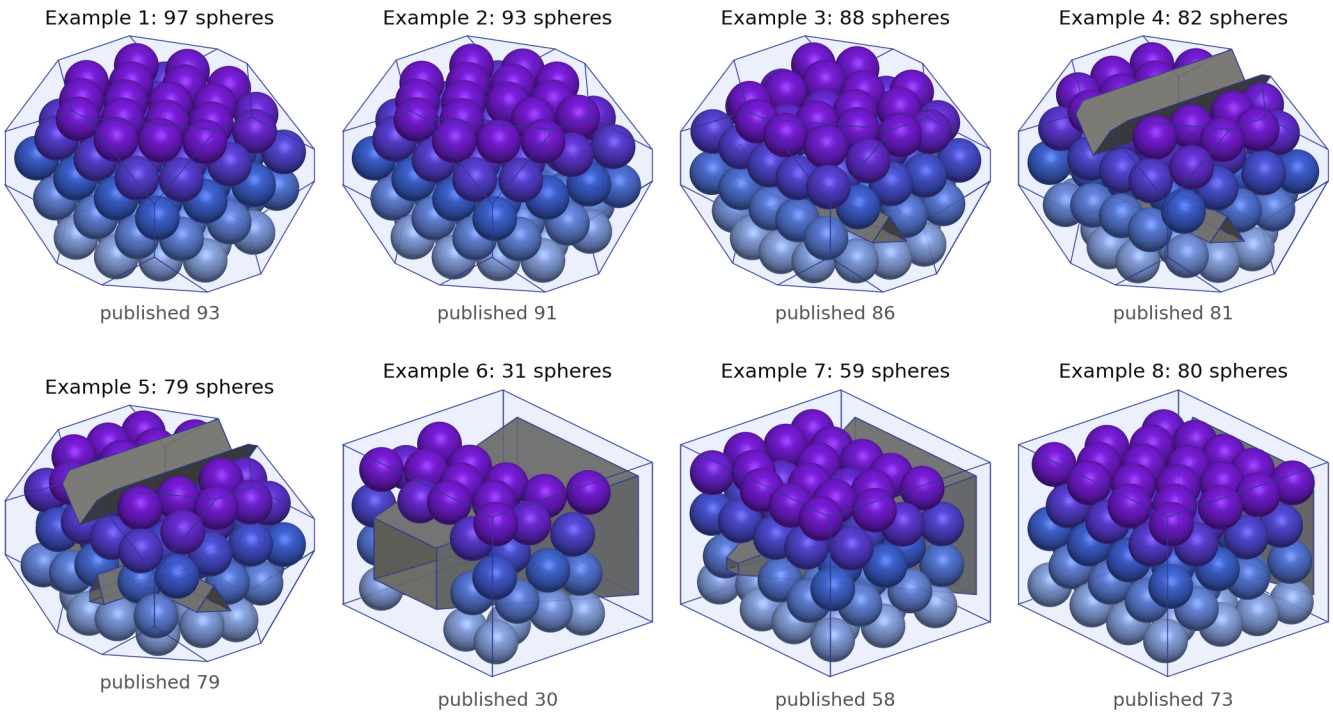


FIG. 17: The most spheres of radius one we fit into each domain of Stoyan and Yaskov [8], best seed of that count; their published count below. The removed pieces (prisms, cones, dihedral angles and the tapered holes of examples 6 to 8) are drawn in grey, and spheres are shaded by height. The centres of these and every other configuration we report are in the repository, with a script that checks them.

in the wall is infeasible and must move. Every trapped run escaped: none ends with a sphere in the wall, and their radii rose from 0.93364 to between 0.968 and 1.007 (Fig. 19). The median radius at each count rose from 0.934 to between 0.978 and 0.998. One more seed fits 30 spheres, but none fits 32, so the escape phase removes the trap without raising our count of 31. All but one of the 50 continuations stopped at the 15-minute limit, before patience ran out, so a longer phase may raise these radii further.

The same domains under ℓ_1 and ℓ_∞ have no published counterpart[keep or cut], and behave as the volumes of the balls predict: at the published counts our best radius is 1.17 to 1.36 under ℓ_1, whose ball is an octahedron, and every count tried fits, while under ℓ_∞, whose ball is a cube, it is 0.85 to 0.89 and none fits.

G. Comparison with General-Purpose Solvers

A referee's first question about a specialised method is whether a general-purpose solver, handed the same problem, would do as well.[tone] We answer it under a protocol fixed in writing before any run, which also committed us to reporting every cell, including those we lose.

a. Solvers and formulation. The baselines are IPOPT 3.14, an interior-point method [24], and SLSQP, sequential quadratic programming [25, 26]. Each is given the constrained program $\max D$ subject to $d(c_i, c_j) \geq 2D$ and a clearance of at least D for every point, formulated as a practitioner would pose it. Dispersion is $\|c_i - c_j\|_2^2 \geq 4D^2$ under ℓ_2; under ℓ_1 and ℓ_∞, whose norms are not differentiable, it uses a smooth approximation of width 10^{-6} times the container's extent. On the convex containers, containment is one exact linear constraint per face, $\langle n_f, c_i - P_f \rangle \geq D \, \|n_f\|_*$. On the non-convex ones it is a signed clearance oracle, the exact distance to the boundary inside and its negative outside, with an analytic gradient, so that a solver always has a direction back into the container. Both solvers receive analytic Jacobians, are restarted from random configurations until their budget ends, and never see our energy model or search.

b. Budget and cells. The 45 cells are every container and metric at $p \in \{10, 20, 40\}$. Each solver makes 10 trials per cell, one core each, and every trial on a cell receives the same wall time, $\max\{60\,\text{s}, 2t_{\text{med}}\}$, where t_{med} is our algorithm's median run time on that cell in the main experiment. A start or local solve still running when the budget ends does not count. Every radius, for every solver, is scored as certified by the bound $B(X)$ of Equation 4.4.

c. Quality. At equal time our algorithm finds a larger radius (best of 10) than IPOPT in 32 of the 45 cells and than SLSQP in 25, and ties the rest to within 10^{-6}

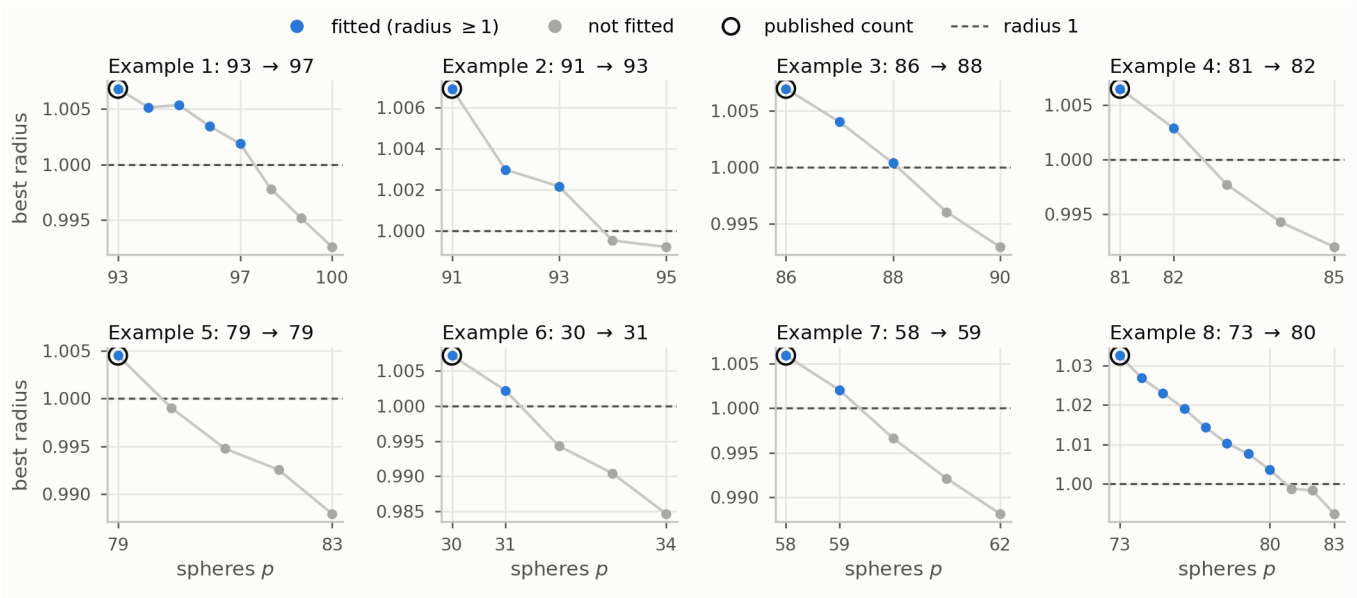


FIG. 18: Best certified radius over 10 seeds at each sphere count tried, in the eight domains of Stoyan and Yaskov [8], under ℓ_2. A count is fitted (blue) when the radius reaches one (dashed); the ring marks their published count, and the panel title gives their count and ours.

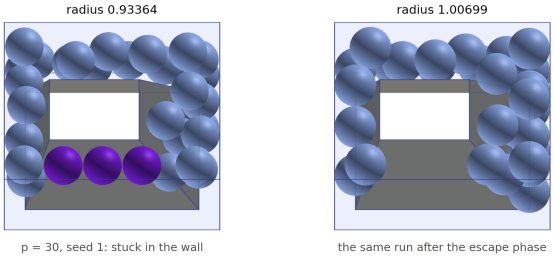


FIG. 19: Example 6 of Stoyan and Yaskov [8], seen end-on along the tapered hole (grey; the far opening is white). Left: a run with three spheres (purple) in the wall under the hole, which caps the radius at 0.93364. Right: the same run continued with the escape phase, the wall empty and the radius 1.00699.

(relative); it is behind in none. A two-sided Wilcoxon signed-rank test on the paired best radii gives $p = 1.1 \times 10^{-13}$ against IPOPT and $p = 3.3 \times 10^{-4}$ against SLSQP. The advantage follows the geometry (Fig. 20): on the convex containers at $p = 10$ all three solvers reach the same radius, and our margin there is at most 5.3% at $p = 20$ and 40, while on the non-convex containers it is ahead by up to 2.6× IPOPT (star, ℓ_2, $p = 40$) and 1.6× SLSQP (cage, ℓ_1, $p = 40$). In every non-convex container the largest gap is at $p = 40$.

d. Speed. The general-purpose solvers are faster where they succeed. In the 8 cells all three solvers solve, all of them convex with $p \leq 20$, the median trial reaches 99.9% of the best radius found 1.5–13× sooner with SLSQP, and up to 3.9× sooner with IPOPT, which is slower than our algorithm in some of them. Our algorithm pays for its ladder start and Tabu Search before its first result: in one of its 450 trials the first start was still running when the budget ended, and such a trial scores zero. Its advantage is reach, not speed. In 19 cells only our algorithm reaches the target in most trials; the reverse never happens, and in 7 cells no solver does. Fig. 22 puts both facts in one picture. SLSQP is the first to the target in 9 cells and reaches it in 18, 9 of them after our algorithm: when it succeeds it is fast, and otherwise it stalls below the target. Our algorithm is first in 29 cells and reaches the target later in 9 more, 38 in all; IPOPT reaches it in 9 and is never first. Fig. 21 shows the same pattern over time on four hard cells: the baselines rise quickly and stop, while our algorithm starts later and keeps improving.

e. Feasibility of reported radii. A local solver stops when its constraint violation falls below a tolerance, so the radius it reports can exceed the radius its configuration supports. Of SLSQP's converged local solves, 36.5% overstate the radius by more than 10^{-6} (relative), 0.9% by more than 1%, and 1.0% report convergence on a configuration that supports no positive radius at all. For IPOPT the shares are 18.3% and under 0.01%. Every radius in this comparison, for every solver, is therefore the certified $B(X)$; our algorithm reports $B(X)$ by construction.

f. Optimality. For the convex containers under ℓ_2, Gurobi's spatial branch-and-bound [23], warm-started from our best configuration, closes its optimality gap within one hour for the cube with $p \leq 9$ and the tetrahedron with $p \leq 10$. In every one of these cases our radius is within 1.5×10^{-5} of Gurobi's bound, which is the resolution of Gurobi's constraint tolerance. For the

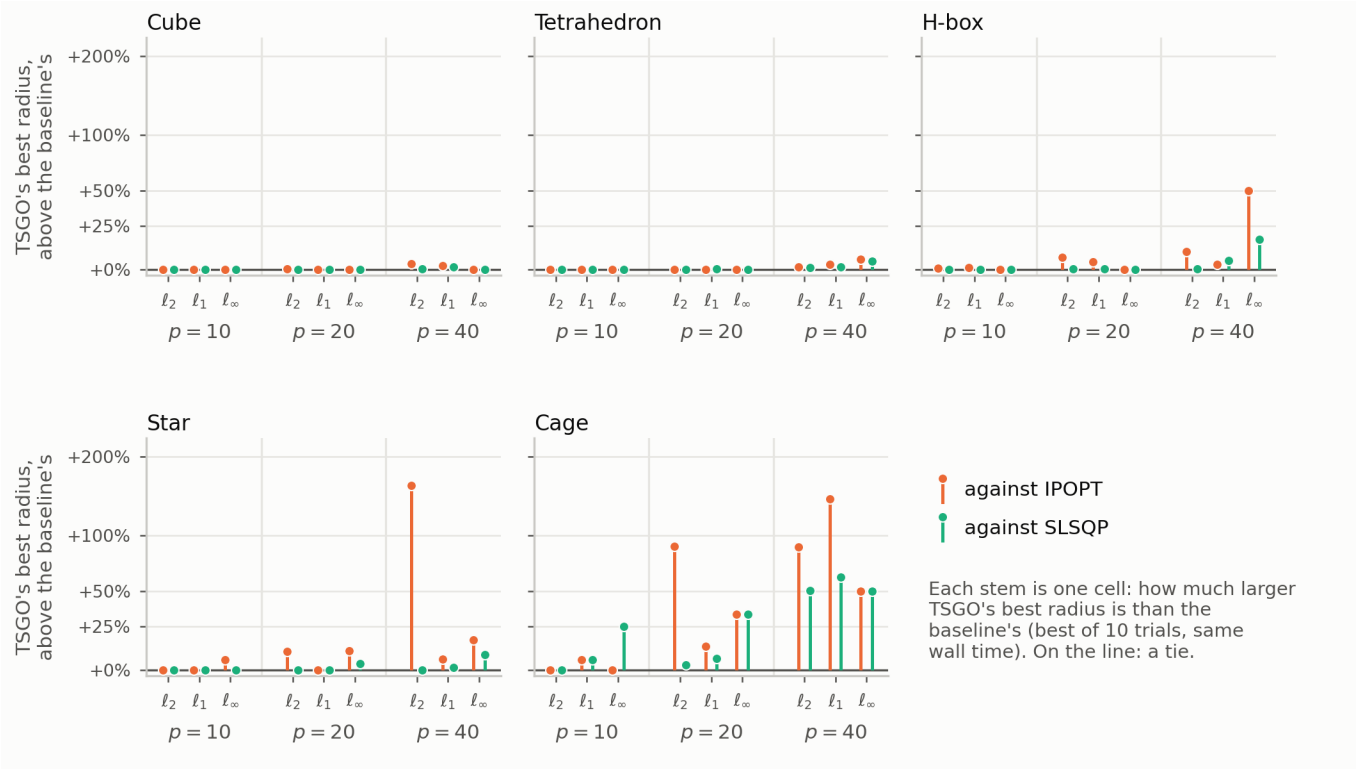


FIG. 20: How much larger our best radius is than each baseline's, in every cell: best of 10 trials at equal wall time. One stem per cell and baseline (IPOPT orange, SLSQP green); a stem's height is our advantage, and a dot on the line is a tie. Within each panel the cells run $p = 10, 20, 40$, and within each p the three metrics. The vertical scale is logarithmic.

cube at $p = 10$, where Gurobi does not close its gap in an hour, Packomania records a proven optimum of $3/14$, which our radius matches to the relative 10^{-9} by which every reported radius is shrunk. On the tetrahedron at $p = 10$ the proven optimum is 0.112372; the benchmark of Kazakov et al. [7], 0.102, is 9.2% below it.

Gurobi also entered the equal-budget comparison on the six cells it can model (cube and tetrahedron, ℓ_2, $p = 10, 20, 40$), on one core per trial like the other solvers, scored by the certified radius of its best incumbent. It ties our algorithm at $p = 10$ and on the cube at $p = 40$, and is behind elsewhere: by 1.3% on the cube at $p = 20$, and by 7.2% and 3.8% on the tetrahedron at $p = 20$ and 40; at $p = 20$ our algorithm reaches the radius of the stacked tetrahedral arrangement, $1/(6 + 2\sqrt{6})$.

H. Computational Efficiency and Scalability

To evaluate scalability, we recorded the wall-clock time of every run, all of its starts and any fallback attempts included ($\epsilon = 10^{-8}$). Fig. 23 plots, for every container and p, the mean over the twenty runs of a cell on logarithmic axes, where a power law is a straight line and its exponent is the slope.

The 30-minute limit shapes the right of the plot. It ended 599 of the 4,800 runs, all at $p \geq 20$ and two thirds of them at $p \geq 50$, so the mean times there measure the limit as much as the algorithm, and the curves of the cube, H-box and star flatten as they approach it. Fitting $\log t$ against $\log p$ over $p \geq 10$ gives exponents of 2.32 for the cube ($R^2 = 0.90$) and 2.06 for the H-box ($R^2 = 0.78$), consistent with the $\mathcal{O}(p^2)$ pairwise distance checks and the $\mathcal{O}(pM)$ boundary checks, where M is the number of faces, and 1.39 for the star ($R^2 = 0.81$), whose curve the limit flattens from $p = 40$. The tetrahedron (1.31, $R^2 = 0.41$) and the ribbed cage (0.14, $R^2 = 0.10$) do not follow a power law. On the tetrahedron no Euclidean run reaches the limit, and the time is set less by p than by how soon the restarts stop raising the radius, which ends a run after $\lceil 3 \ln p \rceil$ restarts without a gain: the mean is 32 to 60 seconds at the tetrahedral numbers $p = 35, 56$ and 84, and 930 seconds at $p = 40$. On the cage the cost is set mostly by feasibility recovery inside a domain whose free volume is broken into narrow channels; it reaches the limit by $p = 30$, and its mean falls beyond $p = 40$.

No container shows growth faster than polynomial up to the limit. The slowest cell, the star under ℓ_1 at $p = 100$, averages 2,121 seconds per run: its runs reach the limit and then finish the restart under way.

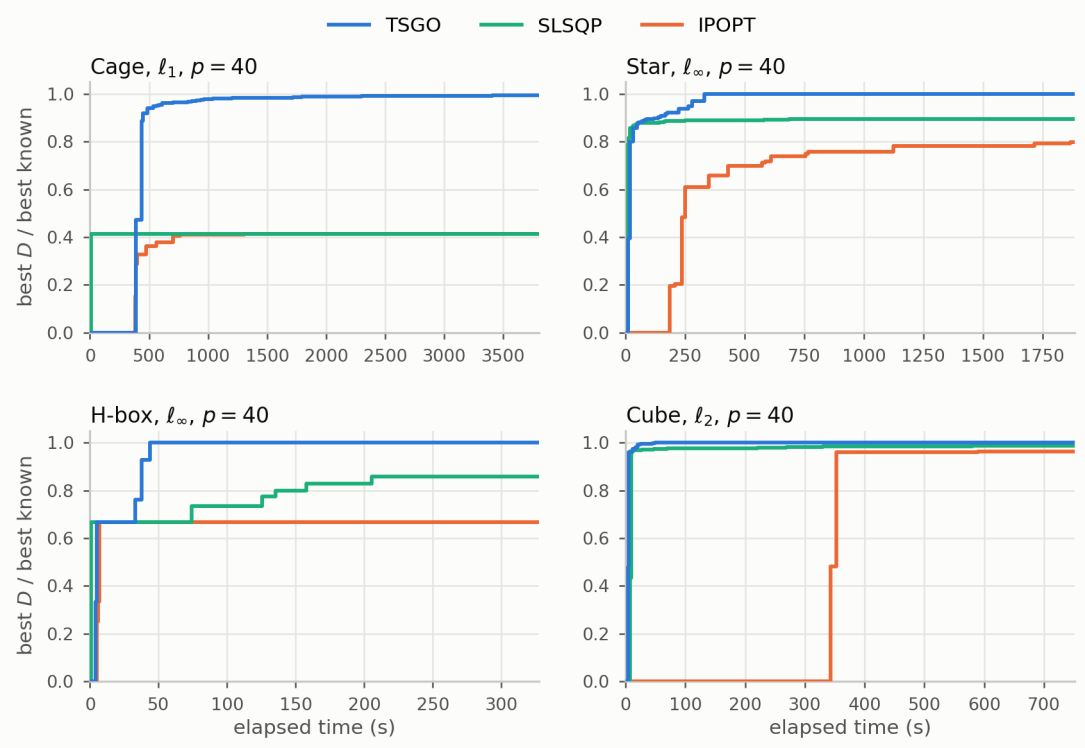


FIG. 21: Best certified radius against elapsed time on four hard cells, as a fraction of the best radius known on the cell (Packomania for the cube); median over 10 trials.

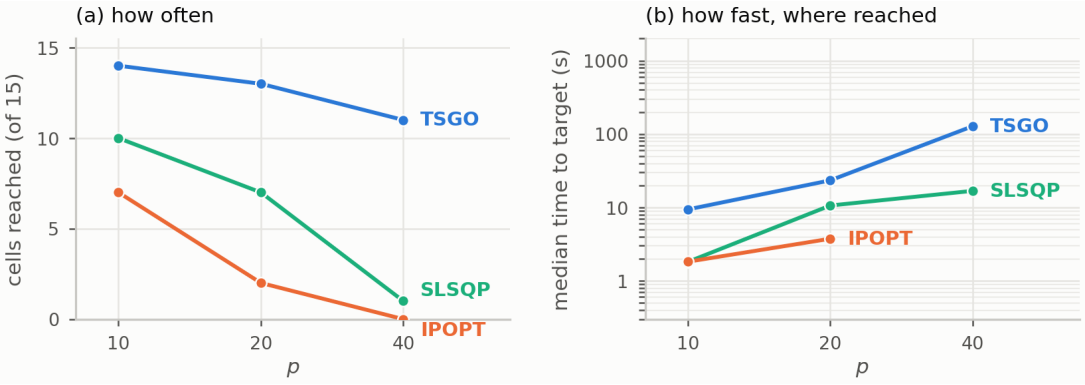


FIG. 22: Reaching 99.9% of the best radius found, judged by each solver's median trial, over the 15 cells (five containers, three metrics) at each p of the comparison panel. (a) The number of cells in which the median trial reaches the target within the budget. (b) The median time it takes, over those cells only; a solver with no such cell at some p has no point there.

VI. CONCLUSION

This paper introduces a robust global optimization framework for the Continuous p-Dispersion Problem (CpDP) in three-dimensional polyhedral containers under any norm-induced metric. By defining an almost-everywhere differentiable feasibility-residual function and extending the Tabu Search algorithm [6], we successfully transition from restrictive two-dimensional planar checks to a comprehensive three-dimensional formulation utilizing a ray-casting procedure and projection-based distance calculations to faces, edges, and vertices. Empirical evaluations confirm that our method matches the proven optima and best-known values in the unit cube [12, 13]; reaches the optima Gurobi certifies in the regular tetrahedron, which the grid-based values of Kazakov et al. [7] miss by up to 9.2%; packs more spheres than published in seven of the eight multi-connected domains of Stoyan and Yaskov [8]; and, at equal wall time, finds a smaller radius than IPOPT or SLSQP in none of 45 cells. It also

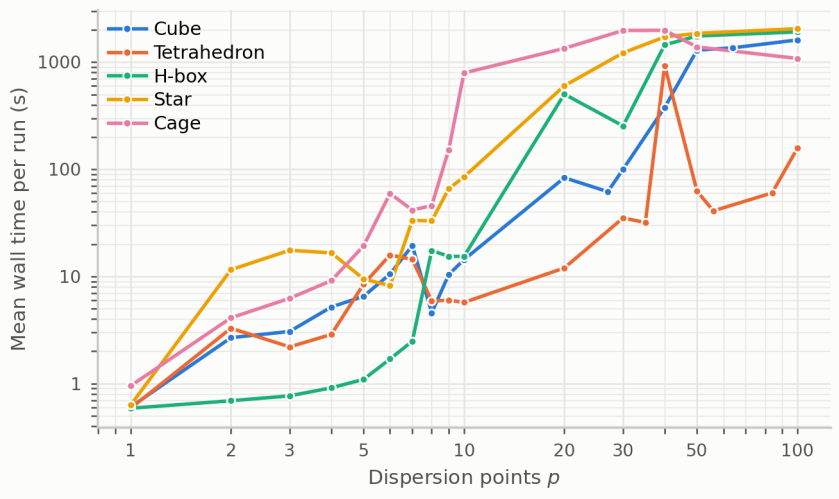


FIG. 23: **Mean solve time per run, Euclidean metric.** Each point is the mean wall-clock time over the seeds of one cell; both axes are logarithmic. The ribbed cage is included but does not scale with p; see the text.

sets benchmarks for three non-convex containers of our own design, with every configuration's centres published alongside a script that checks them, offering a powerful tool for real-world applications such as robotic collision avoidance and facility layout [9, 10].

Restarting at the best radius has a blind spot: a random start already feasible at that radius is never moved, and in three of our cells this held runs in place. The escape phase of Section V 0 a removed it where we applied it, but we chose those cells after seeing them fail; evaluating it across the whole benchmark, and deciding whether it belongs in the default algorithm, is future work.

Because our energy formulation relies on dimension-independent vector operations, it provides a natural foundation for extension into n-dimensional spaces ($\mathbb{R}^n$). The primary adaptation required for higher dimensions is generalizing the geometric container representation, specifically determining how to precisely model $(n-1)$-dimensional bounding hyperplanes and extending the ray-casting validity checks. Furthermore, future work will focus on incorporating more sophisticated, non-planar geometries such as curved surfaces. While polyhedral distances currently admit closed forms — orthogonal projection in the Euclidean case, and a finite search over the dual unit ball's vertices for a general polyhedral norm — curved boundaries require representations like spline patches or implicit surfaces, necessitating the development of computationally efficient distance evaluation methods that do not rely on closed-form projections.

ACKNOWLEDGEMENTS

[!][revtex widetext; C&OR also wants funding, CRediT, data and AI-use statements] The authors would like to thank Ram Padmanabhan for their valuable feedback and editorial assistance throughout the preparation of this manuscript.

[1] Michael Kuby. Programming models for facility dispersion: The p-dispersion and maxisum dispersion problems. *Geographical Analysis*, 19:315–329, 1987.

[2] Zvi Drezner and Erhan Erkut. Solving the continuous p-dispersion problem using non-linear programming. *Journal of the Operational Research Society*, 46(4):516–520, 1995.

[3] Christoph Baur and Sándor P. Fekete. Approximation of geometric dispersion problems. *Algorithmica*, 30(3): 451–470, 2001.

[4] Ronald L. Graham, Boris D. Lubachevsky, Kari J. Nurmela, and Patric R.J. Östergård. Dense packings of congruent circles in a circle. *Discrete Mathematics*, 181 (1):139–154, 1998.

[5] Zhengguan Dai, Kathleen Xu, and Melkior Ornik. Repulsion-based p-dispersion with distance constraints in non-convex polygons. *Annals of Operations Research*, 307 (1):75–91, 2021.

[6] Xiangjing Lai, Zhenheng Lin, Jin-Kao Hao, and Qinghua Wu. An Efficient Optimization Model and Tabu Search-

Based Global Optimization Approach for the Continuous p-Dispersion Problem. *INFORMS Journal on Computing*, 37, 2024.

[7] Alexander L. Kazakov, Anna A. Lempert, and Trungthanh T. Ta. The sphere packing problem into bounded containers in three-dimension non-euclidean space. *IFAC-PapersOnLine*, 51(32):782–787, 2018.

[8] Yuri G. Stoyan and Georgiy N. Yaskov. Packing congruent spheres into a multi-connected polyhedral domain. *International Transactions in Operational Research*, 20 (1):79–99, 2013.

[9] Alaa Z. Abdulghafoor and Efstathios Bakolas. Distributed coverage control of multi-agent networks with guaranteed collision avoidance in cluttered environments. *IFAC-PapersOnLine*, 54(20):771–776, 2021.

[10] Vigneash Loganathan, Saminathan Veerappan, Premkumar Manoharan, and Bizuwork Derebew. Optimizing drone-based iot base stations in 6g networks using the quasi-opposition-based lemurs optimization algorithm. *International Journal of Computational Intelligence Systems*, 17:218, 2024.

[11] Fred W. Glover and Manuel Laguna. *Tabu Search*. Springer, 1997.

[12] Jonathan Schaer. On the densest packing of spheres in a cube. *Canadian Mathematical Bulletin*, 9(3):265–270, 1966.

[13] Eckard Specht. Packomania: Spheres in a cube. `https://www.packomania.com/scu/scu.html`, 2026. Accessed 30 September 2026.

[14] Maxwell Rosenlicht. *Introduction to Analysis*. Dover Books on Mathematics. Dover Publications, 2012.

[15] Mark de Berg, Otfried Cheong, Marc van Kreveld, and Mark Overmars. *Computational Geometry: Algorithms and Applications*. Springer, 3rd edition, 2008.

[16] James R. Munkres. *Analysis on Manifolds*. Advanced Books Classics. Addison-Wesley, 1991.

[17] Roger Fletcher. *Practical Methods of Optimization*. Wiley, 1987.

[18] Wencheng Wang, Jing Li, and Enhua Wu. 2D point-in-polygon test by classifying edges into layers. *Computers & Graphics*, 29:427–439, 2005.

[19] Denis Horvat. Ray-casting point-in-polyhedron test. 2012.

[20] Lawrence C. Evans and Ronald F. Gariepy. *Measure Theory and Fine Properties of Functions*. CRC Press, revised edition, 2015.

[21] Anthony V. Fiacco and Garth P. McCormick. Computational algorithm for the sequential unconstrained minimization technique for nonlinear programming. *Management Science*, 10(4):601–617, 1964.

[22] Thomas Bäck, Christophe Foussette, and Peter Krause. *Contemporary Evolution Strategies*. Springer, 2013.

[23] Gurobi Optimization, LLC. Gurobi optimizer reference manual. `https://www.gurobi.com`, 2026. Version 13.0.

[24] Andreas Wächter and Lorenz T. Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1):25–57, 2006.

[25] Dieter Kraft. A software package for sequential quadratic programming. Technical Report DFVLR-FB 88-28, DFVLR, Oberpfaffenhofen, Germany, 1988.

[26] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17:261–272, 2020.

[27] H. Groemer. Some basic properties of packing and covering constants. *Discrete & Computational Geometry*, 1(2): 183–193, 1986.

[28] Harold S. M. Coxeter. Star polytopes and the schläfli function $f(\alpha, \beta, \gamma)$. *Elemente der Mathematik*, 44(2):25–36, 1989.

[29] Xin Lu, Lu Wang, Liangbi Wang, and Yao Hu. Heat transfer enhancement of diamond rib mounted in periodic merging chambers of micro channel heat sink. *Micromachines*, 16, 2025.